



- Esame:
- Cos'è un algoritmo?
 - Problema dell'ordinamento:
- Algoritmi che vedremo:
 - InsertionSort:
 - Correttezza:
 - Correttezza InsertionSort:
 - Dimostrazione (per induzione):
 - Dimostrazione correttezza:
 - Valutazione complessità dell'algoritmo:
 - La stabilità:
 - MergeSort:
 - Caso base:
 - Passo induttivo:
 - InsertSort:
 - Stime di somme
- ALBERI BINARI
 - Vari casi:
 - Esempio:
 - Cancellazione:
- BTree
 - Operazioni su BTree
 - Ricerca:
 - Creazione albero vuoto:
 - Cancellazione:
 - RB-Tree vs BTree
- Insiemi Disgiunti
 - Insiemi disgiunti con liste concatenate
 - Weighted Union
 - Insiemi disgiunti con Alberi
- Grafi
 - Grafi Pesati
 - Visite sui Grafi
 - Visita in ampiezza (BFS)
 - $1) \implies 2)$
 - Visita in profondità (DFS)
 - DFS visita in profondità
 - Teorema delle parentesi:
 - Teorema di cammino bianco:
 - Classificazione DFS degli archi di un grafo
 - Componenti fortemente connesse
 - Algoritmo di Kosaraju per le S.C.C.
 - Chiusura transitiva



- Da Prodotto tra matrici
- Grafi Pesati
 - Albero minimo di copertura
 - Sottografi di copertura $E' \subseteq E$
 - Algoritmo di Kruskal
 - Algoritmo di Prim
 - Cammini minimi - di peso minimo
 - Single Source Shortest Path
 - All Pairs Shortest Path
 - Algoritmo di Floyd-Warshall



Esame:

Parte scritta orale e laboratorio (laboratorio va fatto prima dell'orale) (probabilmente l'orale è obbligatorio)

5 appelli:

- 2 compitini + orale (fine febbraio e inizio metà giugno, orale giugno o luglio)
- II app con scritto in luglio
- III metà/fine luglio
- IV scritto settembre
- V scritto gennaio/febbraio

Per l'orale serve aver preso almeno 16 nello scritto.

Nei compitini nella media viene arrotondata per eccesso

Fare male il primo compitino non preclude il secondo

Tutte le altre informazioni sono nella pagina e-learning (ancora da fare) Si trovano:

- Le regole d'esame
- Alcuni esami scritti passati
- Qualche esercizio risolto
- Quiz di autovalutazione (vengono pubblicati dopo ogni argomenti, molto più semplici degli esami)
- Qualche link utile a siti per fare simulazioni con alcune strutture dati
- Riferimenti bibliografici
- Tutte le registrazioni e trascrizioni durante le lezioni dell'anno durante la pandemia

Teams viene usato solo per le emergenze

L'altro professore usa di più Teams

Il ricevimento è su appuntamento tramite mail (è sospeso i 7 giorni lavorativi precedenti a uno scritto).

Il libro di testo che si segue è "cormen leiserson Rivest (...) Introduction to Algorithms"

3 requisiti per l'esame

- Programmazione del primo anno (soprattutto per la ricorsione)
- Matematica discreta e analisi (non c'è un blocco, ma è molto consigliato)

Il laboratorio erano esercizi obbligatori da consegnare e un progetto (forse il progetto verrà cambiato) è obbligatorio lavorare in gruppo



Cos'è un algoritmo?

Una sequenza ordinata di operazioni → modelli di calcolo (che tipo di macchina si utilizza) Si deve ragionare su correttezza e complessità e si deve trovare un problema da risolvere. Per risolvere un problema si deve:

avere una funzione che dice cosa ho in input e cosa ho in output.

P: input → output

A: input → output

Le due funzioni devono coincidere e quindi devo dimostrare che quello che ho scritto è quello che deve fare.

Nel corso creeremo dei problemi, creeremo degli algoritmi per risolverli e verrà valutato il tempo per risolverli.

Il modello di calcolo che useremo sarà pseudo codice (molto facile da tradurre in un linguaggio qualsiasi)



Problema dell'ordinamento:

problema: dato un vettore A contenente n numeri interi, determinare una permutazione π degli indici di A tale che

$$\forall i, j, i < j \implies A[\pi[i]] < A[\pi[j]]$$

A vettore Struttura dati statica (dimensione fissa) composta da elementi omogenei, è indicizzato (accesso diretto)

Cos'è una permutazione? (uno scambio tra gli indici – una funzione iniettiva) Un vettore A di lunghezza n ha un indice che va da 1 a n. In input avremo il vettore di interi.

Gli input in questo problema sono tutti i possibili vettori e la dimensione dell'input è la lunghezza del vettore.

Per calcolare la complessità-in tempo (A, M): $\forall \text{ input} \rightarrow \mathbb{R}^+$ Questo processo sarebbe particolarmente complicato, quindi si farà riferimento al caso peggiore, al caso medio e al caso migliore.

$T_p(A, M): \mathbb{N} \text{ (di dimensione } n) \rightarrow \mathbb{R}^+$

Tempo nel caso peggiore: $T_p(A, M)(5)$ quanto tempo richiede nel caso peggiore l'algoritmo A su un generico vettore di dimensione 5 sulla macchina M.

Calcoleremo anche la complessità nel caso migliore.

Ogni operazione di base ha un costo costante c (Che cambia di macchina in macchina), si ragiona in termini asintotici, in modo che non si specifichi questo tempo c, e si prende la funzione che tende $+\infty$



Algoritmi che vedremo:

Su cambi e confronti: **InsertionSort**, **MergeSort**, **HeapSort**, **QuickSort** di cui valuteremo:

- complessità, scambi e confronti
- algoritmi con ipotesi aggiuntive nell'input

Vedremo anche:

- algoritmi sul calcolo della mediana e altre statistiche
- liste, pile, code, code con priorità,
- tabelle di Hash
- Alberi binari di ricerca
- Alberi bilanciati
- B-alberi
- Insiemi disgiunti
- Grafi (non pesati e pesati)



InsertionSort:

Idea:

I numeri che vengono ordinati vengono anche chiamati **keys** (chiavi)

Il metodo è simile a quello intuitivo con cui si ordina un mazzo di carte:



```
InsertionSort(a) {  
    // j parte da 2 perché il primo elemento (a[1]) è già "ordinato" per  
    definizione
```



```

for j <- 2 to a.length {

    key <- a[j]          // Salviamo il VALORE da inserire
    i <- j - 1           // Iniziamo a confrontare con l'elemento a sinistra

    // Sposta gli elementi di a[1..j-1] che sono più grandi di key
    // di una posizione verso destra
    while (i > 0 and a[i] > key) {
        a[i + 1] <- a[i]
        i <- i - 1
    }

    // Inserisce key nella posizione corretta
    a[i + 1] <- key
}
}

```

Dopo aver verificato che il codice funziona con un caso “semplice”, dobbiamo controllare la correttezza e la complessità.

Per controllare la correttezza, dobbiamo definire la funzione con dei passi di computazione, che vanno dall'input all'output.

Correttezza:

P: input $\rightarrow$ output

A: input $\rightarrow$ output (sequenza di istruzioni che termina, $\forall$ input)

Come primissima cosa va dimostrato che termina sempre, gli algoritmi ci permettono di affrontare argomenti decidibili (che si può capire la risposta).

Correttezza InsertionSort:

$\forall$ vettore A InsertionSort (A) termina e al termine A è ordinato in ordine crescente.

$\forall A, b \in [1, A - length] a < b \implies A[a] \leq A[b]$

(e A contiene gli stessi elementi che contiene inizialmente)

Fare attenzione che la correttezza va controllata all'interno del codice, non dell'idea iniziale

La correttezza è il teorema, mentre l'invariante (in questo caso il ciclo for) è come se fosse un lemma ed è il primo elemento che va controllato per la dimostrazione. Nella invariante va codificato matematicamente tutte le proprietà che credo siano vere tutte le volte che passo per la riga di codice. All'inizio della J-esima iterazione del ciclo for $A[1, \dots, j-1]$ è ordinato

Dimostrazione (per induzione):

Base:

$j=2$ $A[1, \dots, j-1]=A[1, \dots, 1]$ un vettore che contiene un solo elemento è un vettore ordinato.

**Passo induttivo:**

ipotesi induttiva: invariante vera all'inizio dell'iterazione j $A[1, \dots, j-1]$ è ordinato

Tesi:

invariante vera all'inizio dell'invariante $j+1$

All'inizio dell'iterazione $j+1$ $A[1, \dots, j]$ è ordinato

Devo quindi analizzare il codice e vedere cosa succede durante la j -esima iterazione del "for"

Un nuovo varole (chiamato x) viene salvato in $key=x$

Quindi $i=x-1$

Di conseguenza i si ferma su un valore $y < x$, tutti gli altri elementi vengono copiati a "destra" di una posizione

All'uscita del while:

(ordinato) (ordinato) (ordinato) x y h (ordinato) (ordinato)

Dimostrazione correttezza:

InsertionSort (A) termina.

Il while termina sempre perché all'interno del while l'indice i viene decrementato e una delle guardie del while richiede che $i > 0$. Il ciclo for termina perché j non è stata modificata all'interno del for. J viene aumentata di 1 a ogni iterazione del for e la condizione richiede che $j \leq A.length$

InsertionSort (A) termina quando la testa (o condizione) del ciclo for viene eseguita con $j = A.length + 1$ e il test del for fallisce all'inizio dell'iterazione $A.length+1$

Sfruttando l'invariante:

$A[1, \dots, (A.length + 1)]$ è ordinato e corrisponde ad A

Valutazione complessità dell'algoritmo:

Calcolo caso peggiore:

$Tp : N \implies R+$ // $Tp(n)$: tempo impiegato nel caso peggiore su un vettore di lunghezza n .

Viene utilizzato il criterio di **costo uniforme** (assumo che ogni istruzione di base del nostro linguaggio di programmazione abbia un costo costante = c)

Le **istruzioni di base** sono: operazioni aritmetiche, assegnamenti, incrementi, test booleani, ecc... Questa variabile è indipendente dalla complessità delle singole operazioni svolte (un esempio $1 + 1$ viene considerato come un c , come sarebbe $5432 * 232332$), per questo motivo questo principio non sempre dà un risultato corretto, ma per questo corso è sufficiente per i casi che vedremo (per casi complessi si dovrebbe usare un sistema di calcolo logaritmico – tiene conto il numero di bit utilizzati)

Anche in caso di lunghezza n fissa, il costo può variare per quanto è disordinato un vettore.

Esempio:



$[2\ 1\ 3\ 3] = [15\ 10\ 20\ 12] = c$ // sono disornati nello stesso modo, quindi hanno la stessa complessità
 $[1\ 2\ 3\ 4] \neq [3\ 4\ 2\ 1] \neq c$ // Il caso peggiore è n ordinato in ordine opposto il caso migliore è un vettore già ordinato

Per il caso migliore: $[2, \dots, n-1] = ((n+1)-2)+1 = n$ (numero di volte che il codice viene eseguito) $C \cdot n$
 $C(n-1)$ ($\forall$ linea di codice)

Il while ha 2 controlli booleani:

- $c \implies c \cdot T_j$ (j = numero di volte che il while viene eseguito)
- La riga sotto il while: $\sum_{j=2}^n (c \cdot (T_j - 1))$

sommando tutto:

$$Tp(n) = c \cdot n + 3c(n-1) \sum_{j=2}^n (c \cdot T_j) + 2 \sum_{j=2}^n (c \cdot (T_j - 1))$$

Sostituendo con numeri:

Nel caso più **fortunato**: $T_j = [1]$

Nel caso **peggiore**: $i=[0, \dots, j-1] \implies$ quindi viene eseguito j volte

Ergo:

$$Tp(n) = c \cdot n + 3c(n-1) + \sum_{j=2}^n (c \cdot T_j) + 2 \sum_{j=2}^n (c \cdot (T_j - 1))$$

ricordando la somma di gauss: $\sum_{k=1}^m k = \frac{m(m+1)}{2} = \Theta(m^2)$

Nel caso migliore: $T_j = 1$

$TO(n) = \Theta(n)$ // dovranno essere distinti gli algoritmi nel caso siano inplace o non inplace (se usano spazio ausiliario o no)

La stabilità:

Un algoritmo di ordinamento è stabile se preserva l'ordine iniziale.

(...)	(...)	(...)	5a	(...)	(...)	(...)
(...)	(...)	(...)	(...)	5a	5	(...)

Un esempio è quando abbiamo un insieme di nomi e cognomi (ordinati per cognome), e vogliamo ordinarli anche per nome, è importante che l'ordinamento del cognome rimanga persistente e successivamente venga applicato l'ordinamento per il nome nel caso in cui due cognomi inizino con la stessa lettera.

Nel nostro algoritmo il while è stato scritto con:



```
(...)  
while (i > 0 && a[i] > key){  
    (...)
```

E va quindi analizzato per determinare se sia o meno stabile



MergeSort:

usa una procedura Divide et Impera

- Divido in sottoproblemi
- Risolvo i sottoproblemi (con ricorsione)
- Combino le soluzioni di sottoproblemi per una soluzione del problema generale

Caso base:

Se il vettore ha lunghezza 1 è già ordinato

Passo induttivo:

due sottoproblemi risolti (esempio: tutti i numeri pari ordinati e tutti i numeri dispari ordinati)

Idea:

Abbiamo un vettore da ordinare, lo dividiamo in due metà (ricorsivamente) fino ad arrivare a vettori di lunghezza 1 (caso base), e poi uniamo i vettori ordinati (merge).

(...) P (...) r (...) q (...)

$$r = \frac{q+p}{2}$$

Determiniamo quindi 2 sottoproblemi e 2 sottovettori da riordinare:

Una volta ottenuti i due vettori ordinati, dobbiamo unirli in un unico vettore ordinato (merge - un nuovo DLC di LLCS).

```
MergeSort(A, p, q) {  
  if (p < q) {  
    // Calcola il punto medio (divisione intera)  
    r <- floor((p + q) / 2)  
  
    // Ordina ricorsivamente la prima metà  
    MergeSort(A, p, r)  
  
    // Ordina ricorsivamente la seconda metà  
    MergeSort(A, r + 1, q)  
  
    // Fonde le due metà ordinate  
    Merge(A, p, r, q)  
    // qui andrebbe definita la funzione Merge  
  }  
}
```




Il caso base sussiste qualora il caso base sia $q \leq p$, dove il programma termina perché il vettore è stato ordinato

Esempio:

10(1) 20 5 2(4)

```
MergeSort (A, 1, 4){
  r<-2
  Mergesort(A,1,2) // viene diviso e ne abbiamo due sotto vettori
  r<-1
  Mergesort(A,1,1) // caso base quindi termina (1=1)
  Mergesort(A,2,2) // caso base come sopra
  Viene fatta quindi la chiamata
  Merge (A, 1, 1, 2) (copia prima il più piccolo, e poi il più grande, crea
  quindi un vettore ordinato)
}
```

Mi trovo quindi con A[10,20] che viene unito al resto (che viene ordinato nello stesso modo)

```
MergeSort(A,3,4){
  r<-3
  Mergesort(A,3,3) // caso base
  Mergesort(A,4,4) // caso base
  Merge(A,3,3,4) // unisce i due vettori su un vettore ausiliario B, mettendoci
  prima il numero minore, poi il numero maggiore
}
```

Merge ora prende i vari vettori ordinati, e tramite LLCS copia il più piccolo di ogni vettore, uno dopo l'altro in maniera ricorsiva, fino a quando non terminano entrambi i vettori

Correttezza Merge:

Merge(A, p, r, q) termina e se all'inizio A[p---r] e A[r+1.....q] sono ordinati, allora al termine A[p, ..., q] è ordinato

Correttezza MergeSort:

MergeSort(A, p, q) termina sempre con A[p....q] ordinato

Dimostrazione correttezza MergeSort:

Per induzione sul numero di ricorsione:

Base:

0 chiamate ricorsive: $p \geq q$ A[p....q] ha lunghezza ≤ 1 ed è quindi ordinato

Passo induttivo:

Mergesort(A, p, q) termina con A[p...q] ordinato se vengono effettuate al più m chiamate

Mergesort(A, p', q') termina con A[p'.....q'] ordinato se vengono eseguite m+1 chiamate



Mergesort(...)

$$m + 1 > 0 \Rightarrow p < q \Rightarrow p \leq r < q$$

quindi MergeSort farà al di più $m+1$ chiamate e quindi termina l'ipotesi induttiva con la correttezza di merge

$$\log_e n \in O(n)$$

Prop:

$$\lim_{x \rightarrow \infty} \frac{g(n)}{f(n)} = g(n) \in O(f(n))$$

$$\forall E > 0 \exists M \forall n \geq M \frac{g(n)}{f(n)} \leq E f(n)$$

$$\lim_{x \rightarrow +\infty} \frac{g(n)}{f(n)} = b \implies g(n) \in O = (f(n))(b \neq \infty)$$

Proprietà:

$$g_1(n) \in O(f_1(n)) \vee g_2(n) \in O(f_2(n)) \implies g_1(n) + g_2(n) \in O(f_1(n) + f_2(n))$$

$$f(n) \in O(f(n))$$

$$g(n) \in O(f(n)) \vee f(n) \in O(h(n)) \implies g(n) \in O(h(n))$$

si usa O come limitazione superiore al costo (il costo è al più)

si usa Ω come limitazione inferiore al costo (il costo è almeno)

$$G(n) \in O(f(n)) \iff f(n) \in \Omega(g(n))$$



InsertSort:

$$f(n) = \sum_{i=1}^n i \cdot k \quad k \geq 1 \text{ costante}$$

$$k \in \mathbb{N}$$

$$f(n) = \sum_{i=1}^n i^3 = 1^3 + 2^3 + \dots + n^3 = \Theta(n^4)$$

Stime di somme

$$\sum_{i=1}^n \log i = \Theta(n \log n)$$

$$\sum_{i=1}^n i \cdot k = \Theta(n^k + 1)$$

$$\sum_{i=0}^n g^i = g^0 + \dots + g^n \implies \text{converge}\left(\frac{1}{1-g}\right) \wedge \text{diverge} + \infty$$

Iterazione:

$$T(n) = T(n-1) + b = (T(n-2) + b) + b =$$

$$(T(n-3) + b) + b + b = \dots =$$

$$(T(n-i) + i \cdot b) = \dots$$

$$(T(1) + (n-1) \cdot b) = a + (n-1) \cdot b = (\Theta)(n)$$

albero di ricorsione:

Numero istanza	Dimensione	Costo	Costo liv
1	n	b	b
1	n-1	b	b
1	n-2	b	b
...	...	...	...
1	1	a	a

$$b(n-1) + a = (\Theta)(n)$$

$$T(n) \geq \sum_{i=0}^{n=1} b \cdot n^2 \cdot \left(\left(\frac{1}{3}\right)^1 + \left(\frac{2}{3}\right)^2\right)^i + 2^{\log \frac{2}{3} n} \cdot a$$

↓

$$bn^2 + n^{\log \frac{3}{2} 2 \cdot a} = (\Theta)(n^2)$$

↓

$$T(n) \in \Omega(n^2)$$

SOSTITUZIONE VARIABILE:

Non si usa con:

T(n)= (una serie di t con numeri minori di 1 – delle frazioni di n) con un costo f(n) (a parte le chiamate



ricorsive) – in questo caso si sua l'albero delle chiamate ricorsive

Si usa con:

$$T(n) = \begin{cases} T(g(n)) + T(k(n)) + \dots + \Theta(f(n)) \\ g(n), k(n), \dots \end{cases} \quad \text{non sono lineari}$$

esempio:

$$T(n) = \begin{cases} \Theta(1) & n \leq 2 \\ 2T(\sqrt{n}) + \Theta(\log n) & n > 2 \end{cases}$$



ALBERI BINARI

L'altezza dell'albero è la lunghezza del più lungo cammino + foglia

Con n nodi l'altezza massima dell'albero è $n - 1$ (h appartiene $O(n)$) Quindi h appartiene $\Omega(\log n)$

TBST Se InOrder(T) Stampa in ordine crescente Ricerca (max min chiave prod) Insertion

Per ricerca massimo e minimo: Minimo: quando continuo a scendere a sinistra fino a quando non trovo più nodi

```

BTS-Min(T){
  x<-T.root
  while (x != NIL && x.left != NIL){
    x<-x.left
  }
  return x
} // nel caso peggiore ha complessità (teta)(h), h = altezza albero
// nel caso l'albero sia completamente bilanciato: (teta)(n), n= numero di
nodi
// in termini di spazio: è In place

```

Ricerca di una chiave k:

```

BST-Search(x,k){ // % cerca K nel sottoalbero radicato in x
  If(x= NIL || x.key = k) // mettere x=NIL prima così in caso vero non
implode
  Return x
}else{
  If(x.key > k){
    Return BST.Search(x.left,k)
  }else{
    Return BST-Search(x.right,k)
  }} // nel caso peggiore: come prima (teta) (h || n), NON è in Place
perché ric.

```

Ricerca del Sucessore Dato T BST e dato x (app) T Il successore y di x è il nodo di T che contiene la più piccola chiave più grande della chiave di x (se esiste)

Se il sottoalbero dx di x è vuoto il più giovane antenato di uni x è parte del sottoalbero di x

```

BST-Successor(X){
  If(x.right != NIL){
    Return BST.minimum(x.right)
  }else{
    Y<-x parent
  }
}

```



```

        while(y != NIL && x != y.left){
            x<-y
            Y<- x.parent
        }
    }return y
}

```

INSERIMENTO

T BST è un nuovo nodo con chiave k che non sta in T

```

BST-Insert(T, z) {           // % z.key non appartiene a T
    y <- NIL
    x <- T.root

    // Scende nell'albero per trovare la posizione corretta
    // x è il puntatore di scansione, y è il puntatore al padre (trailing pointer)
    while (x != NIL) {
        y <- x
        if (z.key < x.key) {
            x <- x.left
        } else {
            x <- x.right
        }
    }

    // Inserimento del nodo z
    z.parent <- y

    if (y == NIL) {
        T.root <- z           // L'albero era vuoto
    } else {
        if (z.key < y.key) {
            y.left <- z
        } else {
            y.right <- z
        }
    }
}

```

Rotazioni di alberi bilanciati (cercare di abbassare Altezza di albero):

Prep: se T è un BST al termine dell'operazione di RightRotate(T, x) ho ancora un BST

T viene passato per gestire il caso in cui x sia la radice. Costo (teta)(1) modifico una quantità costante di posti vicino ad x

```

RighthRotate(T,x){
    P<- x.parent
    Y<- x.left
    If(p=NULL){
        y.parentc<-NULL
    }
}

```



```
        T.root<-y
      }else{
        y.parent<-p
        if(x=p.left){
          p.left<-y
        }else{
          p.right<-y
        }
      }
      Z<-y.right
      x.left<-z
      if(z != NIL){
        z.parent<-x
      }
      y.right<-x
      x.parent<-y
    }
  }
```

Serve una strategia per realizzare le rotazioni per mantenere: BTS bilanciati Per gli alberi rosso-neri:

- le foglie sono NIL e black
- ogni nodo Red ha 2 figli black
- per ogni nodo x lungo ogni cammino x-foglie trovo la stessa quantità di nodi black



Vari casi:

Caso fortunato:

- X Red figlio di Red ha zio Black
- X e suo zio sono **opposti**

Caso quasi fortunato:

- X Red figlio di Red ha zio black
- X e suo zio NON sono **opposti**

Caso sfortunato:

- X Red figlio di Red ha zio Red



Esempio:

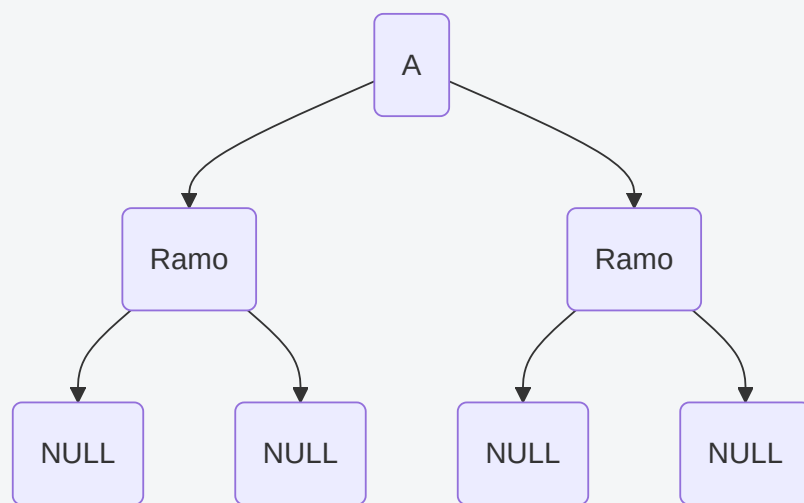
copia le cose dal telefono

Complessità Dato T RBT contenente n chiavi inserire una nuova chiave che costo?

- Secondo $\Theta(\log_n)$
- risalgo con 1, 2, 3 per risalire ($\Theta(1) \cdot O \log_n$)

Totale: $\Theta(\log_n)$

In ogni caso applico al di più 2 rotazioni





Cancellazione:

Standard utilizzati:

Red (rosso)

Black (Blue)

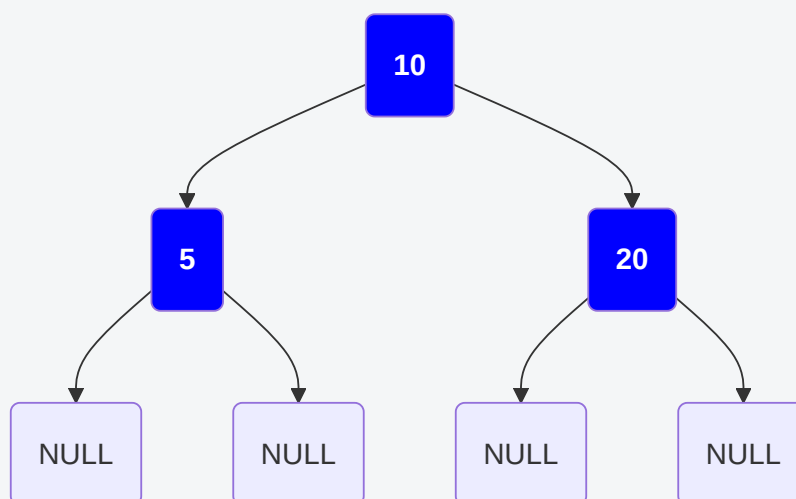
Non Importa

DB (dubble black(Nero))

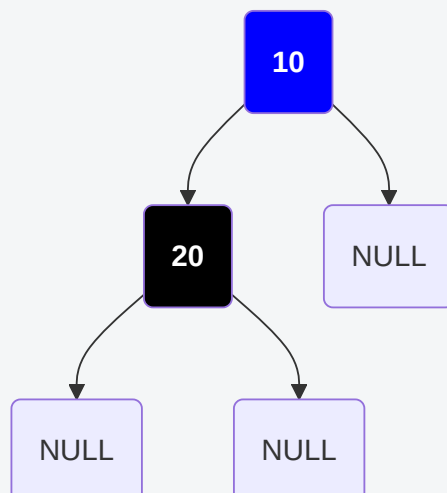
Mi concentro nel nodo che sparisce dall'albero:

Se al suo posto metto o NIL o if il suo unico figlio non NIL

Es:



per cancellare 5:





Dobbiamo tenere conto che 20 ora vale come 2 black

Se a un certo punto della procedura il DB è RED lo ricoloro BLACK e ho finito

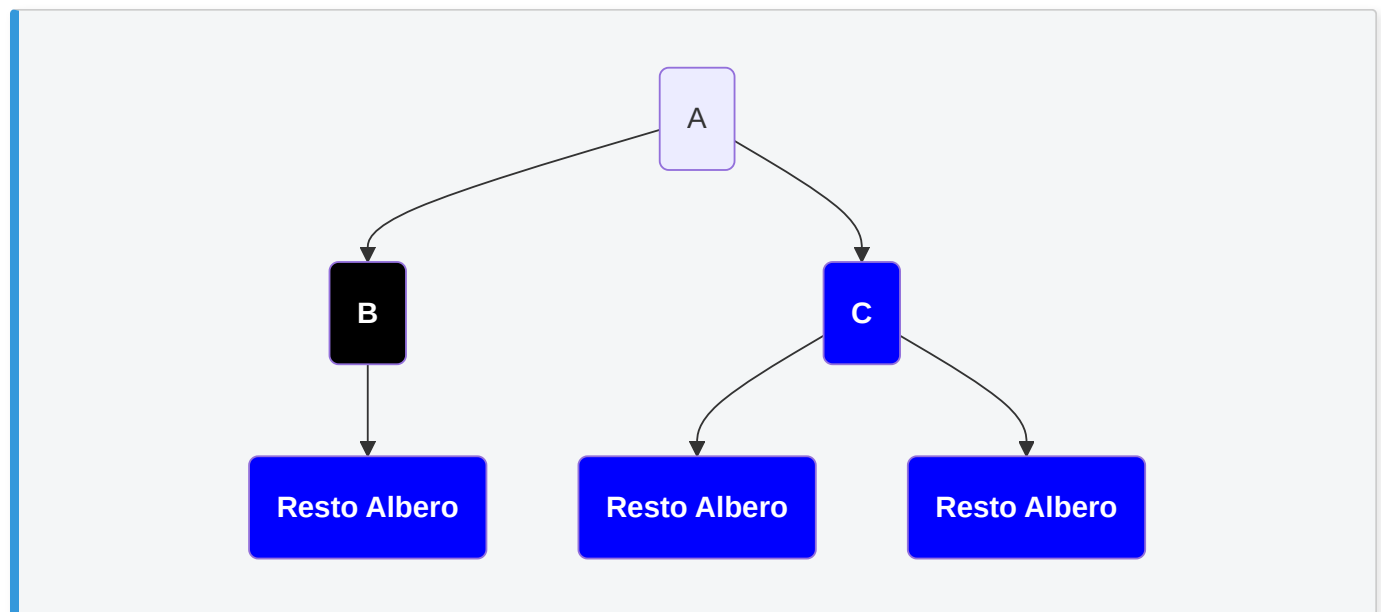
1: Caso fortunato: Le DB (dubble black) ha un nipote Red opposto

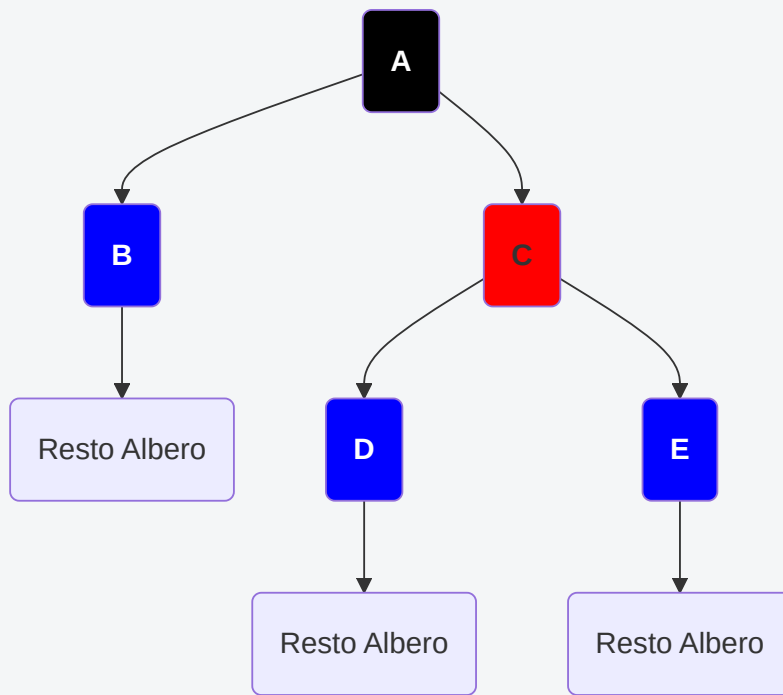
$\text{LeftRotate}(T, p) \rightarrow$ Ricalcolo n forse p e y

2: Caso quasi fortunato: Le DB ha un nipote Red **non** opposto

$\text{RigthRotate}(T, y) \rightarrow$ Caso fortunato (copia appunti)

3: Caso sfortunato:







BTree

Vengono usati soprattutto nei file system dei sistemi operativi e nei database.

Sono la struttura dati più efficienti per lo spostamento di grandi quantità di informazioni, soprattutto tra una memoria centrale a una memoria secondaria.

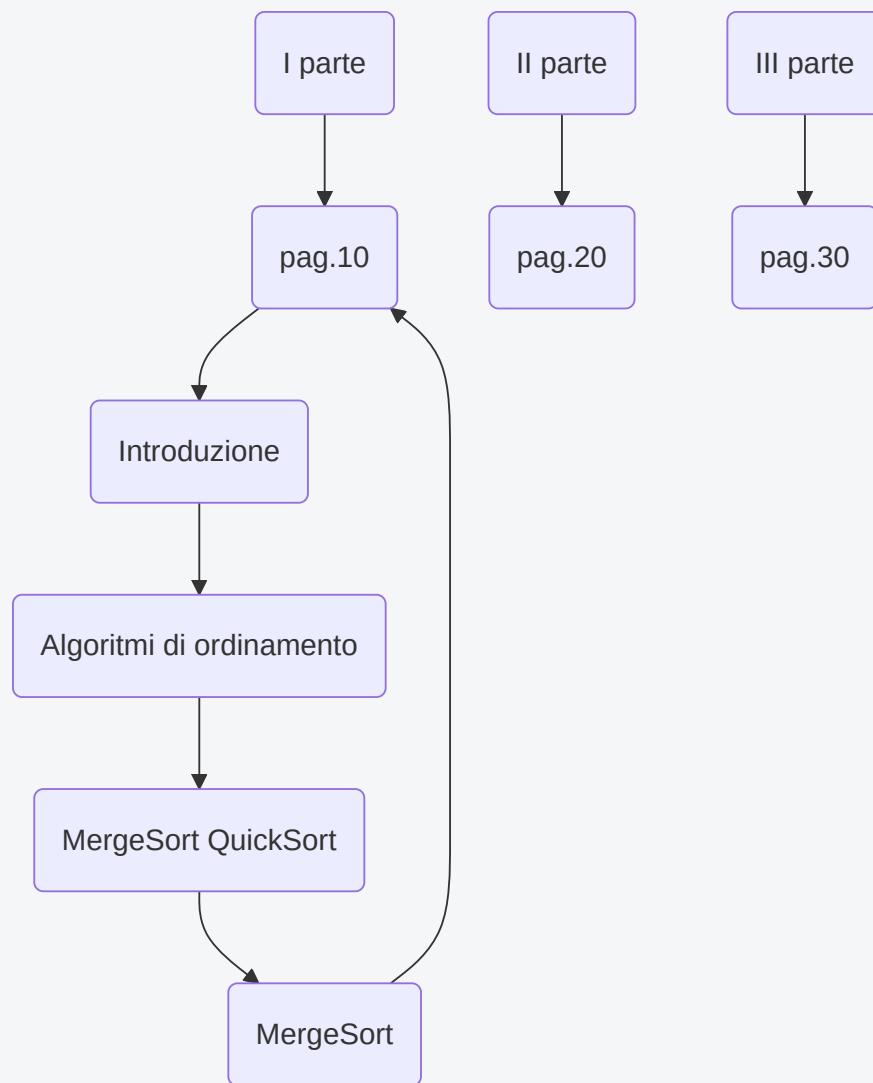


Syntax error in text
mermaid version 11.15.0

- Si cerca di ridurre il numero di R/W (è più lento)
- Si valutano separatamente le complessità in termini di operazioni R/W e CPU

Idea

Generalizza il concetto di **indice**



Definizione

è una struttura dati dinamica composta da nodi in cui:

- ogni nodo x ha un campo Booleano $x.leaf$
- ogni nodo x ha un campo $intro$ x tale che:
 - se x non è la radice $t - 1 \leq x.n \leq 2t - 1$
 - se x è la radice $1 \leq x.n \leq 2t - 1$
 - ogni nodo x ha un campo $x.key$ che è un vettore (di interi) di lunghezza $2t-1$ in cui le informazioni rilevanti sono memorizzate nelle $x.n$ posizioni
 - ogni nodo x ha un campo $x.c$ che è un vettore di puntatori di lunghezza $2t$ in cui se x non è una foglia, le informazioni rilevanti sono memorizzate nelle prime $x.n+1$ posizioni

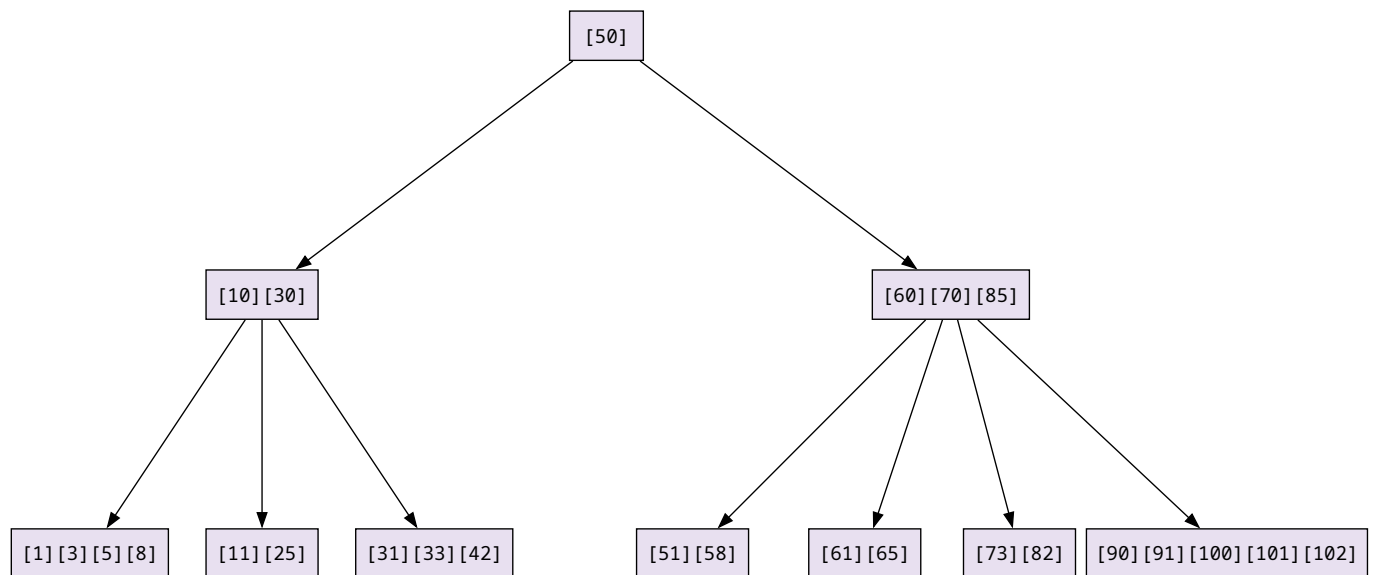
$x.leaf = 0$ $t = 2$ $x.n = 3$ $t - 1 = 0$ $x \neq T.root$ $2t - 1 = 3$

$x.key = [10, 20, 30, _, _]$ $x.c = [\text{puntatore ai figli}]$

- Se y è un nodo appartenente all'albero radicato nell' i -esimo figlio di x allora le sue chiavi sono comprese tra $x.key[i-1]$ e $x.key[i]$
- Tutte le foglie devono trovarsi allo stesso livello (albero bilanciato)

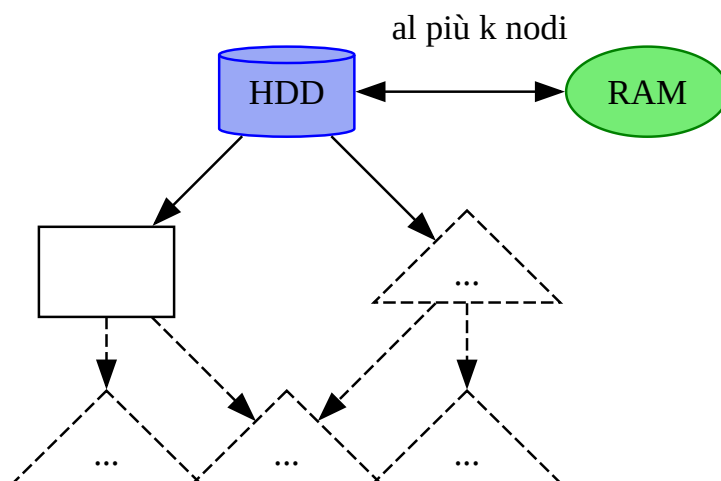


Es.

$$T = 3$$
$$T - 1 = 2$$
$$2t - 1 = 5$$


Convenzioni che useremo:

- No x.percent
 - Variante Corman: Risparmiamo in R/W aumentando in alcuni casi CPU -> Preemptive Split
- Inserimento





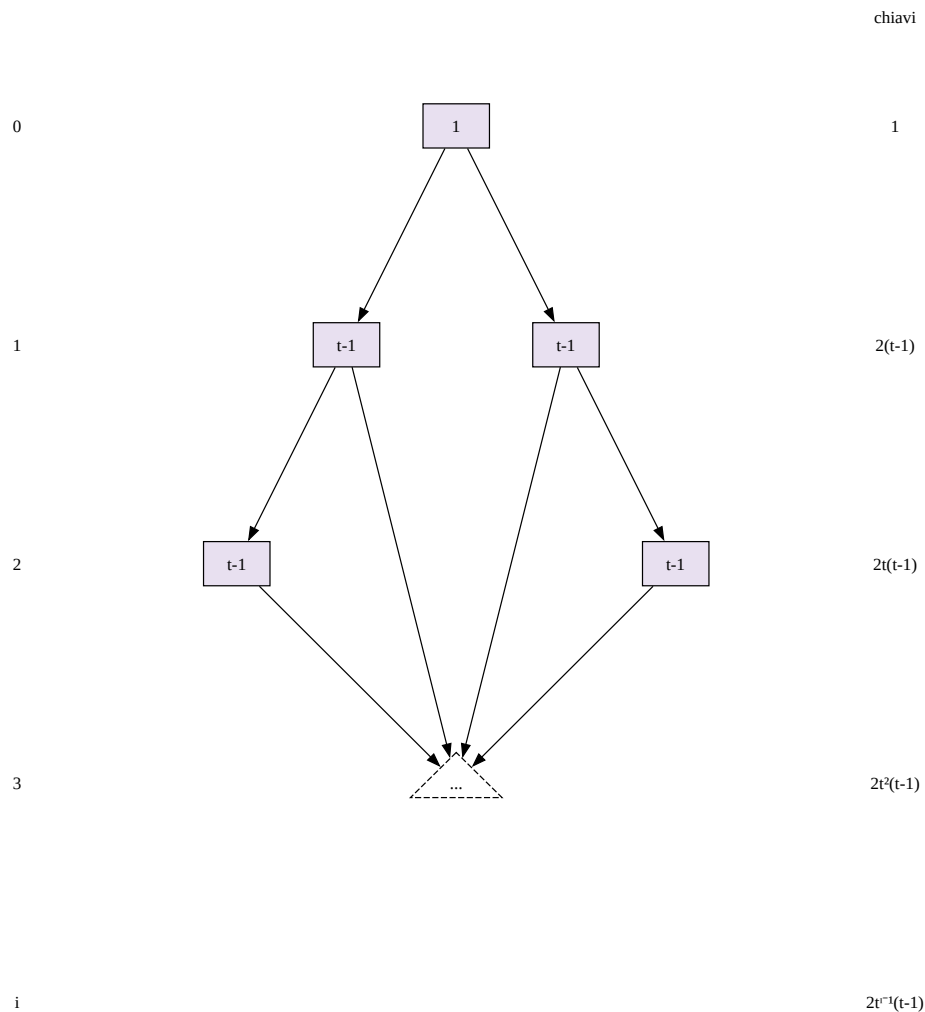
Operazioni su BTree

Ricerca

Inserimento

Cancellazione

Quanto è alto un BTree di grado t contenente n chiavi?



$$\begin{aligned}
 n &= \sum_{i=1}^h 2t \cdot i - 1 + 1 \\
 &= 2(t-1) \sum_{j=0}^{h-1} t^j + 1 \\
 &= 2(t-1) \cdot \frac{t^h - 1}{t - 1} + 1 \\
 &= 2t^h - 1 \implies h = \log_t \left(\frac{n+1}{2} \right)
 \end{aligned}$$



Quindi un BTree di grado t con n chiavi ha una altezza $h \ll \log_t \left(\frac{n+1}{2} \right)$

Quindi: $h \in O(\log_t n)$

Ricerca:

```
BTreeSearch(x, k){      # x in RAM
  i <- 1
  while (i <= x.n && x.key[i] < k){
    i <- i + 1
  }
  if (i <= x.n && x.key[i] = k){
    return (x, i)
  }else if(x.leaf){
    return (NIL, 0)
  }else{
    y <- DiskRead(x.c[i])
    return BTreeSearch(y, k)
  }
}
```

R/W: $O(\log_t n)$

CPU: $O(t \cdot \log_t n)$

PER CASA:

C'è un modo di trasformare un RBT in un BTree?

Del massimo e del minimo

Logicamente il minimo è l'elemento più a sinistra, scendendo a sinistra fino a quando non si trova una foglia

```
# scrivimi
```

complessità

R/W: $O(\log_t n)$

CPU: $O(t \cdot \log_t n)$

Predecessore e Successore

Inserimento in un BTree

Cerco di inserire K in una foglia.

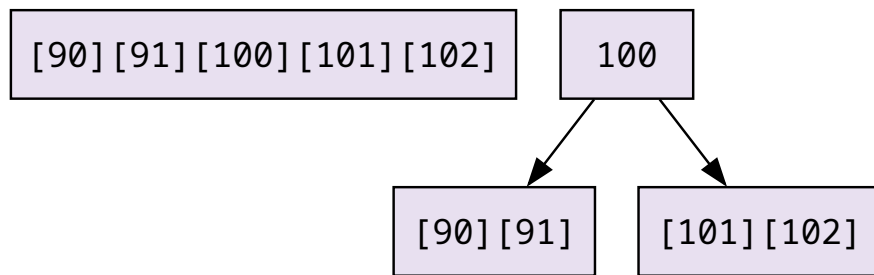
Scendo come se stessi cercando k , inserisco k nella foglia da raggiungere.

Se la foglia è piena (ha $2t - 1$ chiavi) ho un problema

soluzione



possiamo leggere $2t - 1$ come $(t - 1)$, $(t - 1)$, 1 . E quindi ricreo un sottoalberello, con una radice con una sola chiave (il valore centrale) e due figli con $t-1$ chiavi ciascuno.



Posso fare questa operazione solo se y , i -esimo figlio di x , solo se x non ha già $2t-1$ chiavi (x non è pieno)

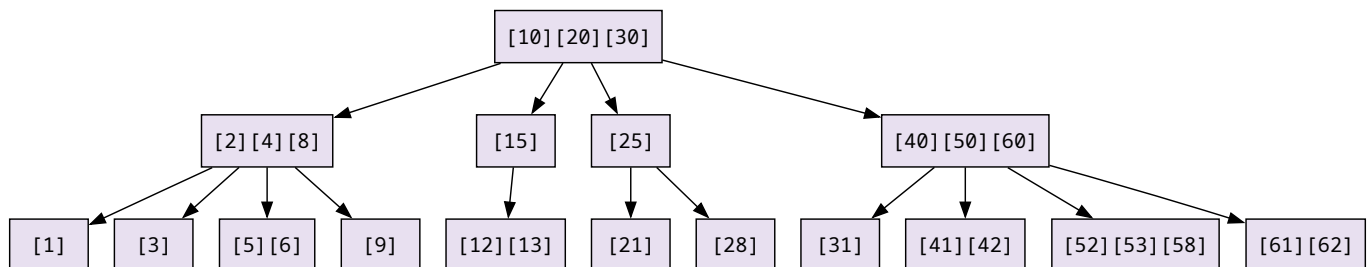
Inserimento con Split Preemptivo

- parto dalla radice
- Scendo come se cercassi k (chiave che devo inserire)
- Se durante la discesa incontro un nodo pieno, lo splitto
- Arrivato a una foglia, inserisco k

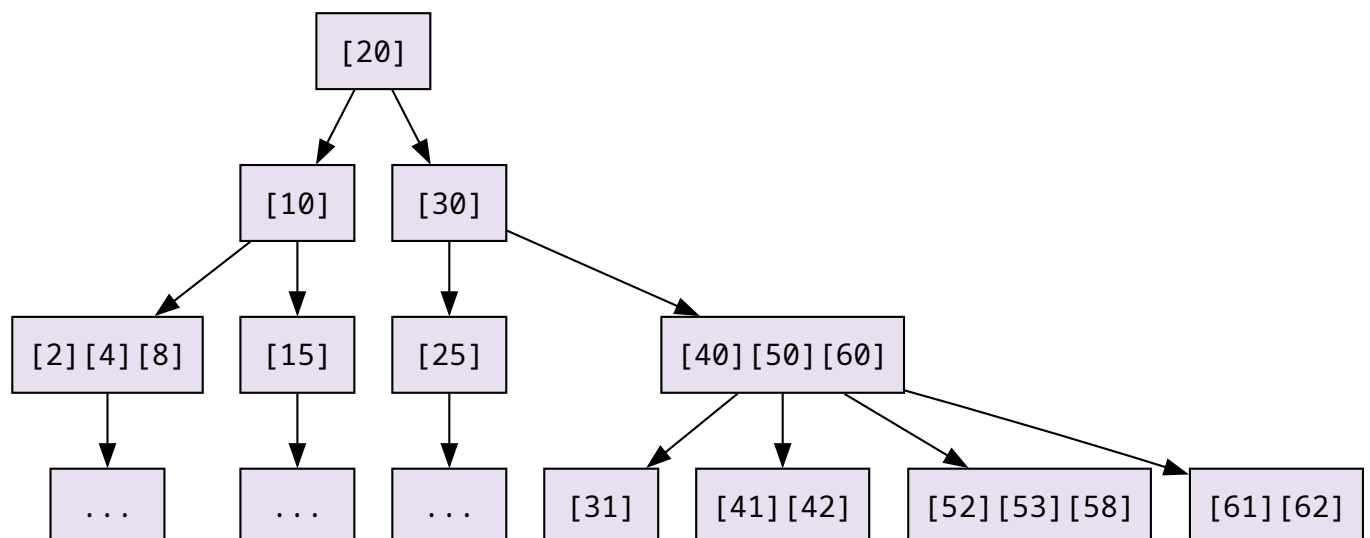
Es.

$t=2$

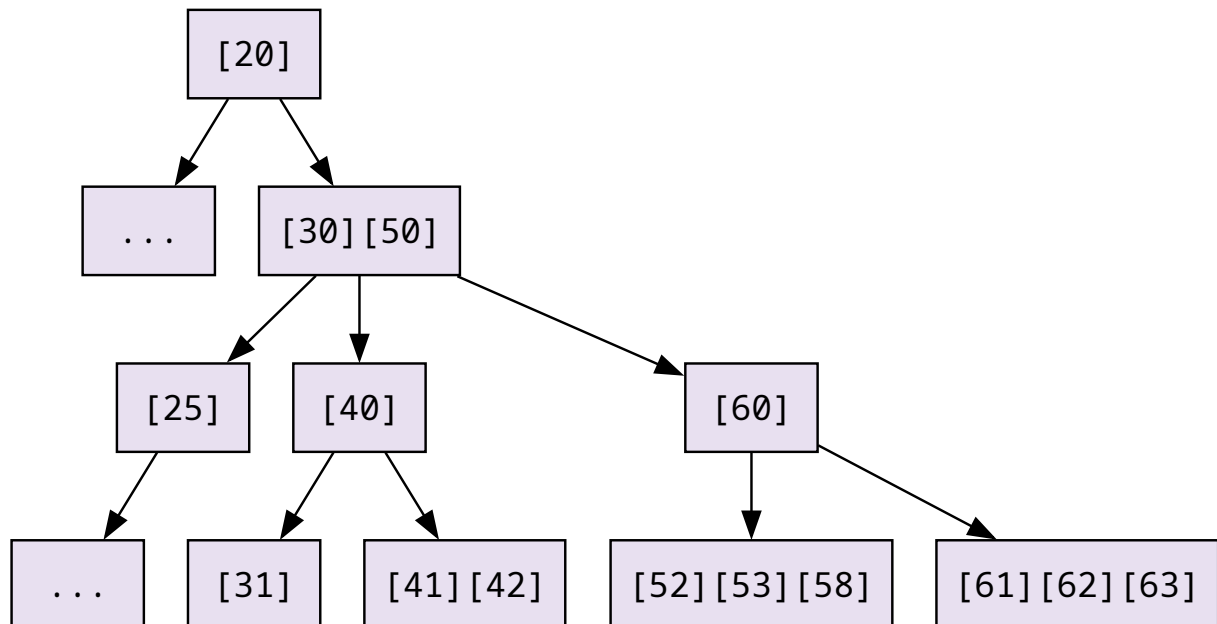
Inserire 1,2,10,5,8,3,9



Quindi diventa:



Continuando a dividere e inserire:





Creazione albero vuoto:

```
BTreeAllocate(t){
  x <- AllocateNode()
  x.n <- 0
  x.leaf <- TRUE
  T.root <- DiskWrite(x)
  return T
}

BTreeSplit(x,i,j,t){      # x non è pieno
  z <- AllocateNode(t)    # y pieno i-esimo figlio di x
  z.leaf <- y.leaf
  z.n <- t-1
  for (j <- 1 to t-1){
    z.key[j] <- y.key[t+j]
  }
  if (not y.leaf){
    for (j <- 1 to t){
      z.c[j] <- y.c[t+j]
    }
  }
  y.n <- t-1
  for (j <- x.n downto i){
    x.key[j+1] <- x.key[j]
  }
  x.key[i] <- y.key[t]
  for (j <- x.n+1 downto i+1){
    x.c[j+1] <- x.c[j]
  }
  x.c[i+1] <- DiskWrite(z)  # andrebbe scritto in memoria secondaria
  x.n <- x.n + 1
  DiskWrite(x)             # modificati in memorie secondarie
  DiskWrite(y)             # nell'indirizzo in cui erano prima
}
```

costi

Indipendentemente da t , R/W è sempre $\theta(1)$, mentre CPU è $O(t)$

Inserimento

```
BTreeInsert(T,t,k){
  r <- DiskRead(T.root)
  if (r.n == 2t -1){
    s <- AllocateNode()
    s.n <- 0
    s.leaf <- FALSE
    s.c[1] <- r
    t.root <- DiskWrite(s)
    BTreeSplit(s,1,r,t)
  }
}
```



```

    }
    r <- DiskRead(T.root) # sicuramete dopo il primo passaggio la radice non è più
piena
    BTreeInsert_not_Full(r,k,t)
  }

```

```

BTreeInsert_not_Full(x,k,t){    # x in RAM, e x non è pieno
  if(x.leaf){
    i <- x.n
    while (i > 0 && x.key[i] > k){
      x.key[i + 1] <- x.key[i]
      i <- i - 1
    } # uscito dal cicolo sono femo
    x.key[i + 1] <- k
    x.n <- x.n + 1
    DiskWrite(x)
  }else{
    i <- 1
    while (i <= x.n && x.key[i] < k){
      i <- i + 1
    }
    y <- DiskRead(x.c[i]) # voglio fare inserimento sul figlio y, ma non
posso se è pieno
    if (y.n < 2t -1){
      BtreeInsert_not_Full(y,k,t)
    }else{
      BtreeSplit(x,i,y,t) # voglio splittare il figlio i-esimo del nodo x
      if (k > x.key[i]){
        y <- DiskRead(x.c[i])
      }else{
        y <- DiskRead(x.c[i+1])
      }
    }
    BtreeInsert_not_Full(y,k,t)
  }
}

```

Cancellazione:

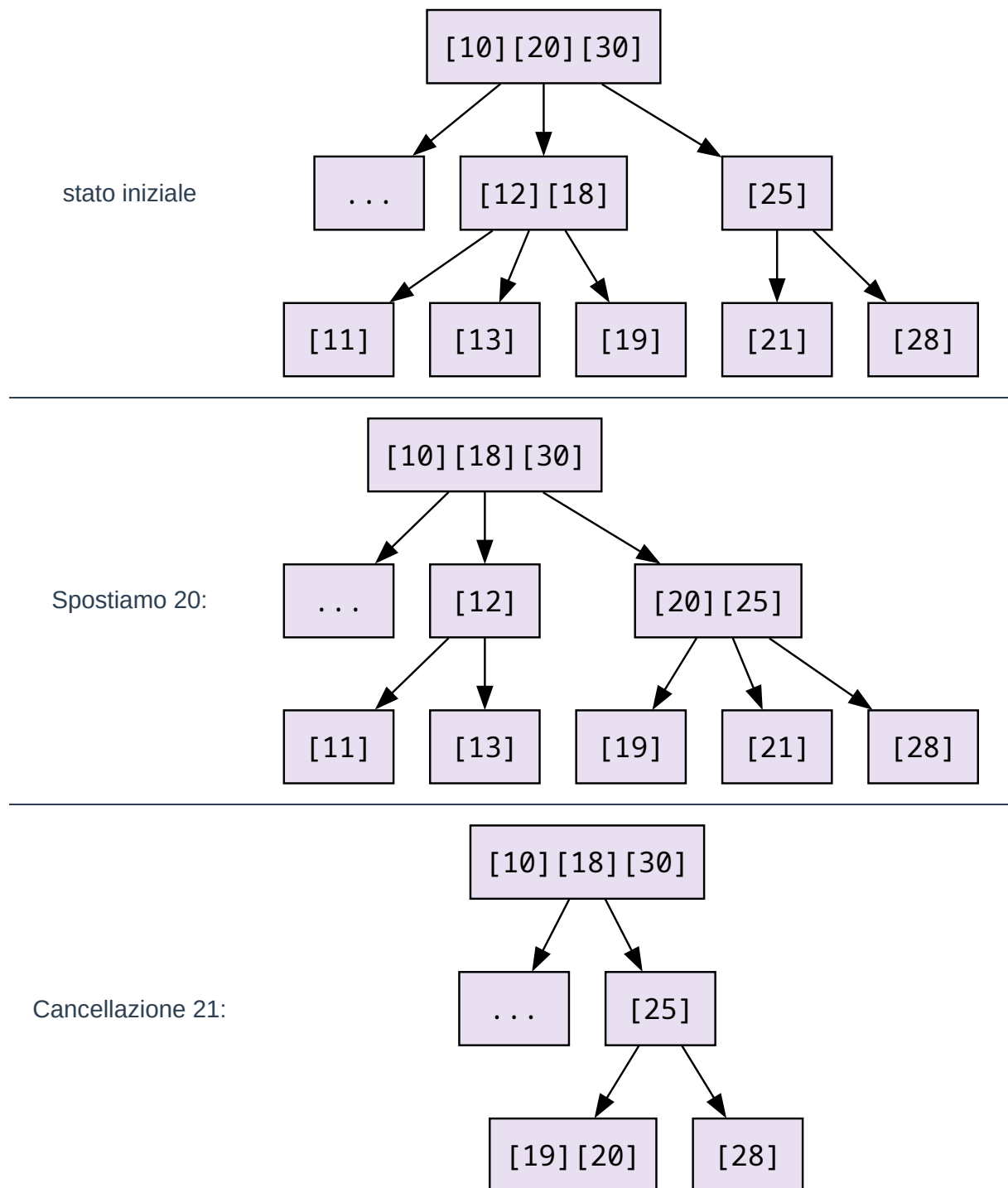
BTree con k da cancellare.

I nodi vuoti ($t-1$ chiavi 1 chiave radice) possono creare problemi. Quindi cerco di sistemare il problema prima di scendere in modo vuoto.

- Inverso dello split se i fratelli sono vuoti
- Rubo una chiave al fratello.

Esempio: $t=2$

e vogliamo cancellare 21:



Se il figlio i ha elemento t chiavi visto il precedente altrimenti il successore

Cancellare k dal sottoalbero radicato in x (x ha almeno t chiavi oppure x è la radice, x è in RAM)

1. Se k è in x e x è una foglia, cancellare k da x
2. Se k è in x e non è una foglia:
 - se y ha almeno t chiavi cancello dal sottoalbero radicato in y il predecessore di k (chiamato ora per convenzione h) e scrivo h al posto di k
 - se y ha solo $t-1$ chiavi e z ha almeno t chiavi, cancello dal sottoalbero radicato in z il successore di k (chiamato ora per convenzione l) e scrivo l al posto di k



- se y e z hanno t-1 chiavi, (non posso scendere perchè vale la regola che ci sono troppe poche chiavi) sposto in giù k e fondo y e z -> w. Cancello k dal sottoalbero radicato in w
3. se k non è in x, trovo il figlio y di x in cui dovrei scendere
- se y ha almeno t chiavi scendo in y e cancello k dal sottoalbero radicato in y
 - se y ha solo t-1 chiavi e il fratello a sinistra di y ha almeno t chiavi, y "ruba" una chiave al suo fratello sinistro e scendo in y e cancello k dal sottoalbero radicato in y
 - se y ha solo t-1 chiavi e il fratello destra di y ha almeno t chiavi, y "ruba" una chiave al suo fratello destro e scendo in y e cancello k dal sottoalbero radicato in y
 - se y ha solo t-1 chiavi e sia il fratello destro che quello sinistro hanno solo t-1 chiavi fondo y con il fratello sinistro e scendo in y e cancello k dal sottoalbero radicato in y

Esempio

t=3

t-1=2

2t-1=5

chiavi da eliminare (in ordine): 36, 16, 60, 51

(fai il diagramma e risolvilò per casa)

La cancellazione Btree ha un costo:

- $\Theta(\log_t n)$ **R/W**
- $O(t \cdot \log_t n)$ **CPU**



RB-Tree vs BTree

- I BTree sono soprattutto usati per gestire una enorme quantità di memoria
- Sono entrambi bilanciati, ma i BTree sono più bilanciati dei RB-Tree

Esercizio per casa: SCRIVERE una procedura che calcoli l'altezza di un BTree



Insiemi Disgiunti

Problema: Gestire K insiemi disgiunti di oggetti $S_1, S_2, \dots, S_K$

$S_i = x_{i1}, x_{i2}, \dots, x_{it} \forall i, j (i \neq j \rightarrow S_i \cap S_j = \emptyset)$ Su cui effettuare le seguenti operazioni:

- **Make(x):** Crea un nuovo insieme disgiunto S_i con un solo elemento $x \rightarrow S = x$
- **Find(x):** Determinare in quale insieme si trova x
- **Union(x, y):** Unire l'insieme che contiene x con l'insieme che contiene y

Valutare il costo di m operazioni M.U.F. (Make, Union, Find) su n elementi di cui m Make

Vedremo due implementazioni:

- **Liste:** Weighted Union (unione pesata)
- **Alberi:** Union by Rank (unione per rango) + Path Compression (compressione del percorso)

Non possiamo usare vettori, perchè non sappiamo in anticipo quanti insiemi disgiunti ci saranno, e quindi non possiamo allocare un vettore di dimensione fissa.



Insiemi disgiunti con liste concatenate

L'idea è quella di mantenere in memoria ogni elemento di un insieme disgiunto, tramite una lista concatenata.

$$\begin{array}{l}
 S_1 \quad \square \rightarrow [x_1] \square \rightarrow [x_1] \square \rightarrow \dots \\
 S_2 \quad \square \rightarrow [x_2] \square \rightarrow [x_2] \square \rightarrow \dots \\
 S_i \quad \square \rightarrow [x_i] \square \rightarrow [x_i] \square \rightarrow \dots \\
 S_K \quad \dots
 \end{array}$$

Make(x):

ottengo un elemento x, già formattato correttamente

Considero x.key già inizializzato, dobbiamo aggiungere solo x.next che punta a NIL

```

Make(x){
  x.next <- NIL
  x.rap <- x // x.rap è un puntatore al rappresentante dell'insieme disgiunto,
che è il primo elemento della lista
  x.last <- x // x.last è un puntatore all'ultimo elemento della lista, che è
utile per l'union pesata
  return x
}

```

Costo: $\Theta(1)$

Find(x):

$$\begin{array}{c}
 x \\
 \hline
 S_3 \quad \square \quad \dots \quad \rightarrow \square \rightarrow \square \rightarrow \dots
 \end{array}$$

Deve restituire un puntatore al 1° elemento della lista in cui si trova x.

Serve un puntatore al 1° elemento della lista

Tutti gli elementi della lista puntano al rappresentante dell'insieme disgiunto, che è il primo elemento della lista

```

Find(x){
  return x.rap
}

```

Costo: $\Theta(1)$

**Union(x, y):**

Come prima cosa controllo che siano due liste distinte, altrimenti non devo fare nulla

```
Union(x,y){
  if (x.rap != y.rap){
    // l'ultimo della lista di x punta al y.rap
    // cambio tutti i rap della lista di y
  }
}
```

Per abbattere il costo della prima operazione, evito di scorrere tutta la lista di x per raggiungere l'ultimo, aggiungo in ogni elemento un puntatore *x.last*.

x.last è aggiornata solo nel rappresentante

```
Union(x,y){
  z <- x.rap
  w <- y.rap
  if (z != w){
    Link(z,w)
    return z
  }
}
```

```
Link(z,w){ // z e w sono i rappresentanti delle due liste da unire
  z.last.next <- w
  z.last <- w.last
  while (w != NIL){
    w.rap <- z
    w <- w.next
  }
}
```

Costo m operazioni di cui m make:

|||Costo string|Costo tot| |---|---|---|---|---| |Make| $n|\Theta(1)|\Theta(n)$ $O(n)$ | |Find| $f|\Theta(1)|\Theta(f)$ $O(f)$ |
|Union| $u \leq n - 1|O(n)|O(n^2)$ | |m=m+f+n|| $O(m+n^2)$ ||

È possibile avere costo $O(m + n^2)$?

Ho creato all'inizio n insiemi disgiunti, dove ogni elemento è sia rappresentante che ultimo.

All'i-esimo passo li unisco una lista lunga i a una lista lunga 1, mettendo per seconda la lista lunga i.

i-esima union costa $\Theta(i)$ costo complessivo:

$$\sum_{i=1}^{n-1} \Theta(i) = \Theta(n^2)$$



Weighted Union

Nelle unioni si mette sempre come seconda la lista più corta

```
W_Union(x,y){
  z <- x.rap
  w <- y.rap
  if (z != w){
    if (z.lenght >= w.lenght){
      z.lenght <- z.lenght + w.lenght
      Link(z,w)
      return z
    }else{
      w.lenght <- w.lenght + z.lenght
      Link(w,z)
      return w
    }
  }
}
```

Proprietà:

M operazioni Make, W_Union, Find su n elementi di cui m Make hanno costo $O(m + n \log n)$

Dimostrazione:

Considero quanto/quando una W_Union comporta un cost nell'oggetto x.

Ho un costo su x solo quando x finisce nella seconda lista, Quante volte x può finire nella seconda lista?

Quando x finisce nella seconda lista modifico $x.rap \rightarrow \Theta(1)$

Ogni volta che x finisce nella seconda lista la lunghezza della lista in cui sta x viene almeno raddoppiata.

x in lista lunga 1 $\rightarrow x$ in lista lunga 2 $\rightarrow x$ in lista lunga 4 $\rightarrow x$ in lista lunga 2^i

Al più $\log n$ volte riesco a modificare $x.rap$

Considerando tutti gli n oggetti:

$$n \cdot \log n = O(n \log n)$$

Esercizio:

Illustrare la famiglia di insiemi disgiunti che si ottiene eseguendo...

Sia 10 insiemi singoletto contenenti le chiavi 1, 2, 3 . . . , 10

Liste con Waited union Nel caso di liste aventi la stessa lunchegga si ...

Union(1,5), Union(3,2), Union(1,2), Union(4,7), Union(7,8), Union(2,8).

Copia grafico

ES:

Gestire insiemi disgiunti di chiavi intere implementando anche l'opzione **Find_Max(x)** che restituisce la chiave monomia tra quelle che trovano nello stesso insieme in cui si trova x.



Insiemi disgiunti con Alberi

Sono alberi con l'unico puntatore che punta al genitore, la radice dell'albero che è l'unica al genitore, ha un solo puntatore a se stesso.

```
Make(x){
  x.parent <- x
  return x
}
```

Costo: $\Theta(1)$

```
Find(x){
  if (x.parent == x){
    return x
  }else{
    return Find(x.parent)
  }
}
```

Costo: $O(h)$, h = altezza dell'albero

```
Union(x,y){
  z <- Find(x)
  w <- Find(y)
  if (z != w){
    Link(z,w)
    return z
  }
  return z
}

Link(z,w){
  w.parent <- z
}
```

Il caso peggiore è $\Theta(n)$?

Sì, se unisco sempre una lista lunga 1 a una lista lunga i , ottengo un albero con altezza $n-1$

|||| |---|---|---|---| |Make| $n \leq m | \Theta(1) | \Theta(n) |$ |Find| $f \leq m | O(n) | O(f \cdot m) \quad O(n \cdot m) |$ |Union| $u \leq n - 1 | O(m) | O(n^2) |$ |m=m+f+n| $| \Theta(n \cdot m) |$

Ottimizzazione: Union by Rank

Aggiorno:



```
Make(x){  
  x.parent <- x  
  x.rank <- 0  
  return x  
}
```

Nulla cambia nella Find, mentre nella Union:

```
Union_by_Rank(x,y){  
  z <- Find(x)  
  w <- Find(y)  
  if (z != w){  
    if (z.rank >= w.rank){  
      Link(z,w)  
      if(z.rank == w.rank){  
        z.rank <- zrank +1  
      }  
      return z  
    }else{  
      Link(w,z)  
      return w  
    }  
  }  
  return z  
}
```

Quanto posso raggiungere al massimo gli alberi?

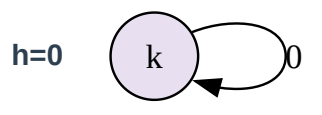
Proprietà:

Alberi di insiemi disgiunti contenenti complessivamente n chiavi (oggetti) e generati usando union by rank hanno altezza $h = O(\log n)$

Dimostrazione:

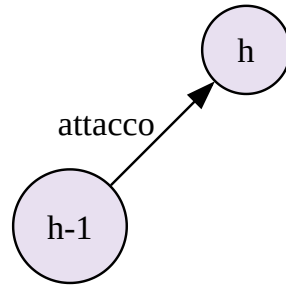
Un albero costruito con union by rank di altezza h contiene almeno 2^h nodi

Base:



Passo Induttivo:

Nelle Union by Rank che hanno portato a ottenere l'albero di altezza h , sicuramente una è stata effettuata su due alberi di altezza $h-1$



Il costo totale di m operazioni è $O(m \cdot \log n)$, usando union by rank

Il costo alto della Find penalizza union by rank, ma esiste la **Path Compression** che consente di trovare un compromesso per abbassare il costo.

Path Compression:

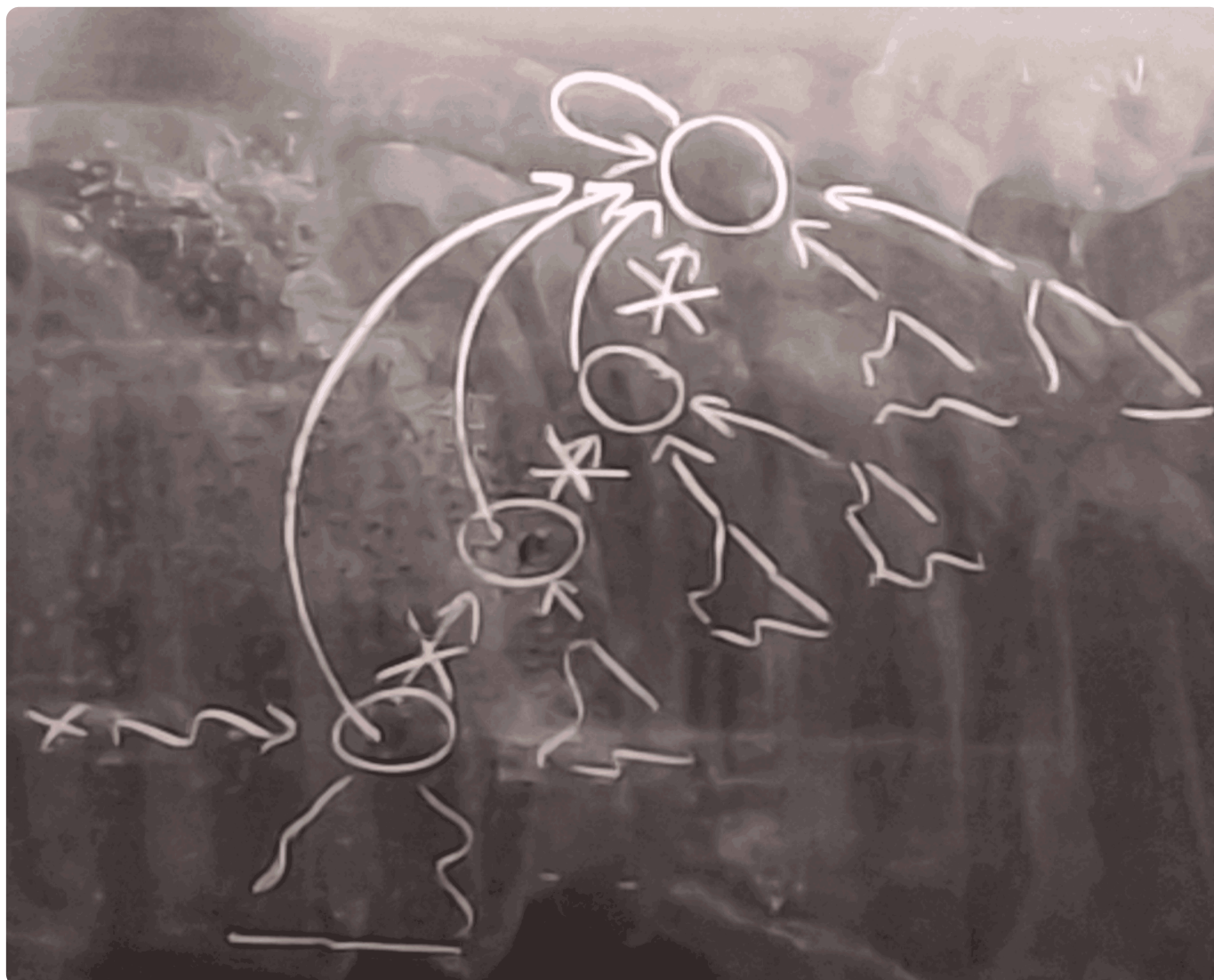


rifallo in paint

```
Find_Path_Compression(x){  
  if (x.parent = x){  
    return x  
  }else{  
    x.parent <- Find_Path_Compression(x.parent)  
    return x.parent  
  }  
}  
  
Find_PC(x){      // versione più elegante ricorsiva
```




```
if (x.parent != x){  
    x.parent <- Find_PC(x.parent)  
}  
return x.parent  
}
```



Con Union by Rank + Path Compression, m operazioni M.U.F. su n elementi di cui m Make hanno costo $O(m \cdot \alpha(m, n))$, detta inversione di Ackermann. Con l'alfa si intende poco più che lineare

Accorciando il percorso, find non è più rapido solo per il nodo x , ma anche per tutti i nodi che si trovano lungo il percorso da x alla radice.

Il costo è quasi lineare perchè solo dopo la costruzione (ogni nodo ha un solo arco, perchè ogni nodo ha un solo genitore, e una volta eseguito il Path Compression l'arco intermedio viene eliminato) accumulo un costo che al più è il costo di Find(x) che ha costo $\Theta(1)$, quindi per tutti è $O(M)$.

Cosa è il rango se uso Union_by_Rank + Path Compression?

E cosa è il rango se non usassi la path compression?

Senza PC il rango sarebbe stato l'altezza dell'albero.

Con PC il rango non lo modifica, ma approssima per eccesso l'altezza dell'albero (costerebbe troppo aggiornare il rango)



Esercizio:

$1, 2, \dots, 10$

Union(1,5), Union(3,2), Union(1,2), Union(4,7), Union(7,8), Union(2,8).

Fare union by rank + path compression.

Finisci grafico



Grafi

Non Pesati:

Abbiamo visto BFS e DFS:

- BFS lunghezza cammini minimi (distanze)
- DFS cicli componenti forti connessi e ordinamento topologico nei grafi che non hanno cicli **Pesati**:
- Calcolo altezza minima di copertura MST
- Calcolo dei cammini di peso minimo



Grafi Pesati

Grafo Orientato/Non Orientato:

Grafo Orientato	Grafo Non Orientato
$G = (V, E)$	$G = (V, E)$
V è un insieme finito di nodi	V è un insieme finito di nodi
$E \subset V \times V$ insieme di archi	E è un insieme di sottoinsiemi di nodi di cardinalità 2
$V = \{1, 2, \dots, V \} \quad V = a$	$V = \{1, 2, \dots, n\}$
$= \{1, 2, \dots, n\}$	$= \{\{1, 2\}, \{3, 4\}, \dots\}$

Matrice di adiacenza:

$$G = (V, E)$$

$$M_G = |V| \times |V|$$

$$M_G[i, j] = \begin{cases} 0 & (i, j) \notin E \\ 1 & (i, j) \in E \end{cases}$$

Esempio:

$$G = (V, E) \text{ con } V = 1, 2, 3, 4$$

La matrice di adiacenza è:

$$M_G = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Proprietà:

Se G è non orientato, M_G è simmetrica e sulla diagonale ho tutti 0

vettore di liste di adiacenza:

Ho un vettore L , che è un vettore di liste concatenate, in cui $L[i]$ è la lista dei nodi raggiungibili da un arco dal nodo i

$L[3] \rightarrow [2] \rightarrow \text{NIL}$ $L[1] \rightarrow \text{NIL}$ $L[2] \rightarrow [3] \rightarrow [4] \rightarrow \text{NIL}$

Quanto spazio occupa?

- Matrice di adiacenza: $\Theta(|V|^2)$
- Vettore di liste di adiacenza: $\Theta(|V| + |E|)$, E = numero di archi

Notiamo subito che $|E| \leq |V|^2$, quindi il vettore di liste di adiacenza è più efficiente per grafi densi.



Ma è vantaggioso come tempo di computazione?

Lo spazio risparmiato è evidente, ora controlliamo se è più efficiente in termini di tempo di computazione

Esempio:

Dato G e dati $u, v \in V$ decidere se $(u, v) \in E$

- Matrice di adiacenza: return $M_G[u, v]$ $O(1)$
- Vettore di liste di adiacenza: in $L_G[u]$ Controllo se compare $O(|L_G[u]|) = O(|V|)$

Esempio:

Dato G e dato un u contare gli archi usati da u

Matrice di adiacenza	Vettore di liste di adiacenza
count <- 0	count <- 0
for(i <- 1 to V) {	x = L_G[u]
if(M_G[u, i] == 1) {	while(x != NIL) {
count <- count + 1	count <- count + 1
}	x <- x.next
}	}
return count	return count

Esempio:

Dato G e dato $u \in V$ contare gli archi in u

Matrice di adiacenza	Vettore di liste di adiacenza
Conto gli 1 nella colonna u : $\Theta(V)$	Scandisco tutte le liste $O(V + E)$ nel caso peggiore: $\Theta(E + V)$

Esempio:

Dato G orientato calcolare il grafo G' in cui gli archi sono invertiti

$$(u, v) \in E \iff (v, u) \in E'$$

Matrice di adiacenza	Vettore di liste di adiacenza
$(M_G)^T$ trasposta $\Theta(V ^2)$	costruisco $L_{G'}$ scandendo tutte le liste di L_G : $\Theta(V + E)$

Esempio:

Dato G orientato calcolare il grafo G' in cui gli archi rappresentano cammini di lunghezza 2 del grafo G

$$(u, v) \in E' \iff \exists z : (u, z) \in E \wedge (z, v) \in E$$

Matrice di adiacenza	Vettore di liste di adiacenza
Prodotto Righe X Colonne tra matrici Booleane	(operazione estremamente complessa e tediosa che la prof non ha letteralmente spiegato perchè troppo lungo)

**Matrice di adiacenza****Vettore di liste di adiacenza**

$M_{G^2} = (M_G)^2 = M_G \cdot M_G$ con costo:
 $n^2 \cdot n = n^3$ con algoritmo Naive

è ancora un problema aperto, e non si sa
 ancora come risolverlo in $\theta(n^2)$

Esempio:

Dato G calcolare G^*

$$(u, v) \in E^* \iff u \rightsquigarrow \dots \rightsquigarrow v$$

M_{G^*} ha la stessa complessità di **1 prodotto** tra matrici

problema:

Dato G orientato, calcolare il grafo G^*

$$G(V, E)$$

$$G^*(V, E^*)$$

$$(u, v) \in E^* \iff u \rightsquigarrow \dots \rightsquigarrow v$$

Cammino:

$$u \rightsquigarrow \dots \rightsquigarrow v$$

$$u \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k \rightarrow v$$

Lunghezza = numero di archi

esempio:

"Mearmind 2 -> 1 2-> 5 1 ->3 5 -> 3

3 -> 7

7-> 4 4 -> 6 6 -> 7

"

" (grafico dove tutto va ovunque) 2 -> 1 2-> 5 2 -> 2 2->3

2-> 7 1->7

1 ->3 5 -> 3 5 -> 5

3 -> 7

7-> 4 4 -> 6 6 -> 7 2->6 "

$$(1, 2) \notin E^*$$

$M_G \rightsquigarrow M_{G^2} = (M_G)^2 \quad M_{G^{\leq 2}} = M_{G^2} + M_G + I$ (sfida per casa: abbassare da $\Theta(|V|)$ a $\Theta(\log |V|)$)

$$M_G \rightsquigarrow M_{G^*} = I1 + M_G + (M_G)^2 + (M_G)^3 + \dots (M_G)^{|V|-1}$$

Quindi: $\Theta(|V|) \cdot \Theta(|V|^\alpha)$



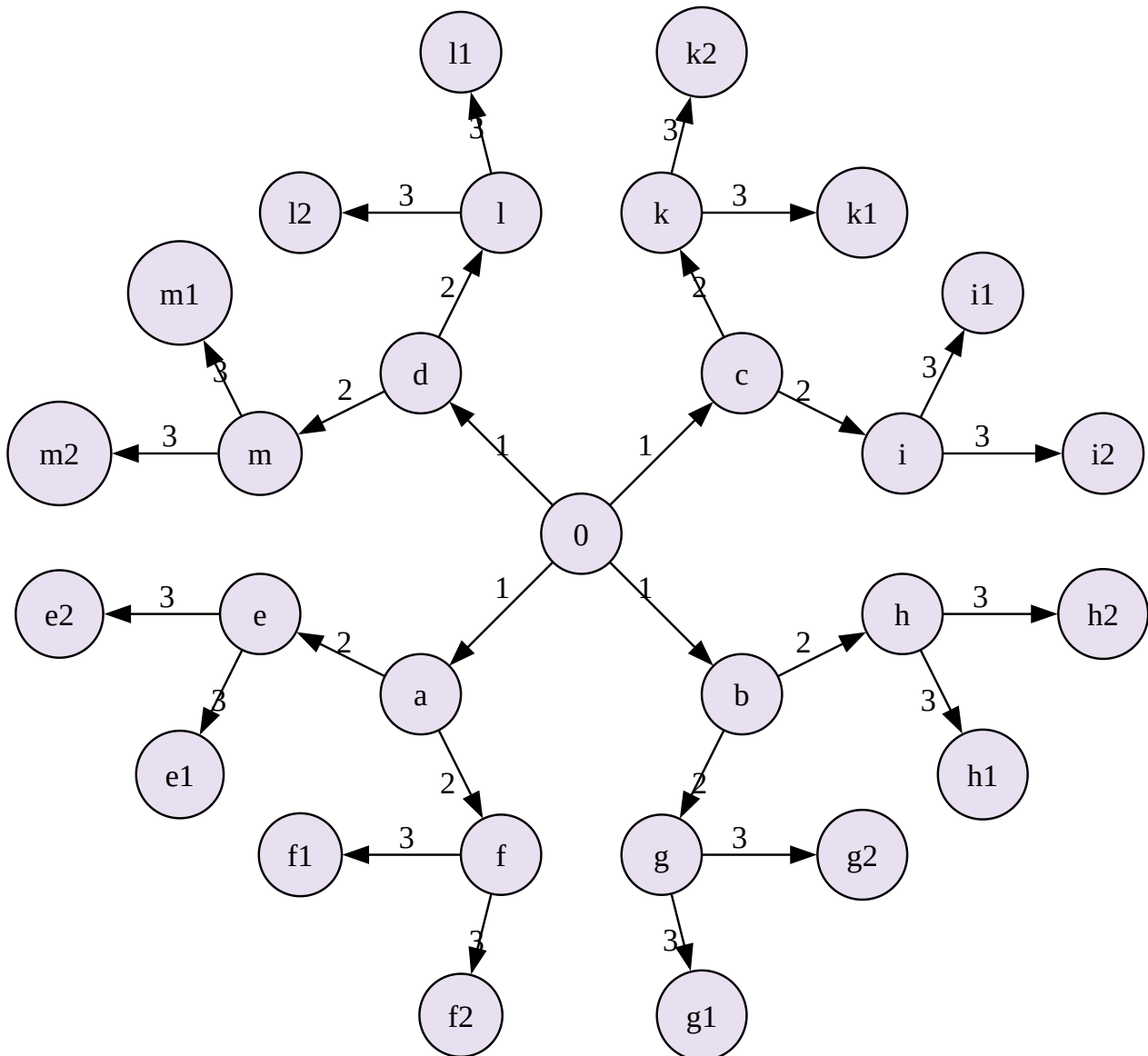
- Come faccio a calcolare M_{G^*} in $\Theta(|V|^\alpha)$?



Visite sui Grafi

Visita in ampiezza (BFS)

Dato un grafo $G = (V, E)$ e un nodo $s \in V$, Esplorare tutti i nodi del grafo che sono raggiungibili da s , andando in ordine di distanza crescente da s



Cos'è la distanza in un grafo?

La distanza tra due nodi u e v è la lunghezza del cammino minimo tra u e v :

$$\delta(u, v) = \begin{cases} +\infty & \text{se } u \not\rightsquigarrow v \text{ in } G \\ n & \text{n lung di un cammino minimo da } u \text{ a } v \end{cases}$$

$$u, v \in V$$

Che priorità deve avere la distanza?

- $\delta(u, v) = 0 \iff u = v$
- $\delta(u, v) \geq 0$



- $\delta(u, v) = \delta(v, u)$ solo nei casi orientati
- $\delta(a, b) \leq \delta(a, c) + \delta(c, b)$

BFS calcola $\delta(s, v) \forall v \in V$ e calcolare un cammino di lunghezza minima da s a v $\forall v \in V$

$G \text{ --- } M_G \text{ oppure } L_G$

grafico che cicla, solo 3 elementi

color di vettore di lung $|V|$

$$color[v] = \begin{cases} \text{Bianco} & \text{non visitato} \\ \text{Grigio} & \text{trovato ma non visitato} \\ \text{Nero} & \text{visitato} \end{cases}$$

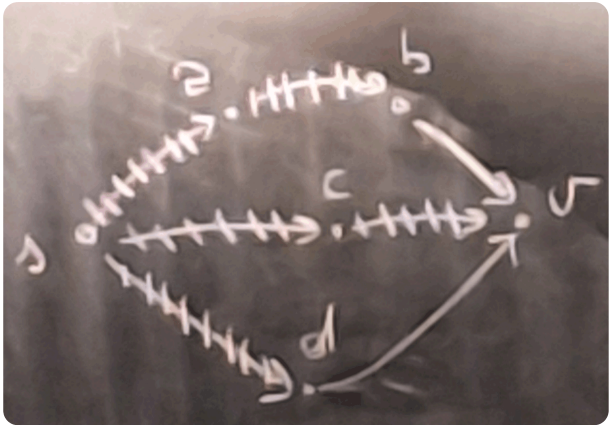


Grafico dei cammini di lunghezza minima da s

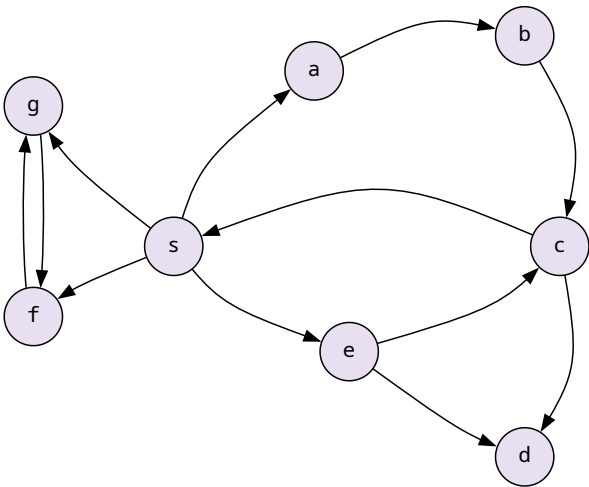


è un albero con radice s

Quindi basta tenere in memoria un vettore π di lunghezza $|V|$, $\pi[v]$ il precedente di v nel cammino minimo da s a v

Scrivere un codice Q per procedere nell'ordine giusto

esempio



s	a	b	c	d	e	f	g
B	B	B	B	B	B	B	
∞	i	∞	∞	∞	∞	∞	∞
0		2		2			
N	S	N	N	e	N	S	S
S	Q	b	c	d	e	d	

$Q = [s, a, e, f, g]$

coloro i vari nodi a distanza di uno di grigio a man mano che li raggiungo



esempio

$$Q = [\cancel{x}, \cancel{Q}, \cancel{e}, \cancel{f}, \cancel{g}, d, c, o']$$

```

BFS(G, s){
  foreach(v in V){
    color[v] <- Bianco
    pi[v] <- NIL
    d[v] <- +\infty
  }
  Q <- NewQueue()
  d[s] <- 0 # distanza da s a s è 0
  color[s] <- Grigio
  Enqueue(Q, s)
  while (a != Empty(Q)){
    u <- Head(a)
    for(v in Adj[u]){ # questo for dipende da come è memorizzato il grafo
      if (color[v] == Bianco){
        color[v] <- Grigio
        pi[v] <- u
        d[v] <- d[u] + 1
        Enqueue(Q, v)
      }
    }
    Dequeue(Q)
    color[u] <- Nero
  }
}

```

Complessità:

Matrici di ADS

Liste di ADS

- Entrano in Q i raggiungibili da s
- Ogni nodo entra in Q al più una volta grazie ai colori

$$O(|V|)^2$$

$$O(|V| + |E|)$$

Correttezza:

BFS(G, s) termina e al termine per ogni $v \in V$:

- $d[v] = \delta(s, v)$
- $\pi[v]$ è il nodo che precede v in un cammino di lunghezza minima da s a v (tra $\delta(s, v)$)

Lemma:

Se $(u, v) \in E$ allora $\delta(s, v) \leq \delta(s, u) + 1$

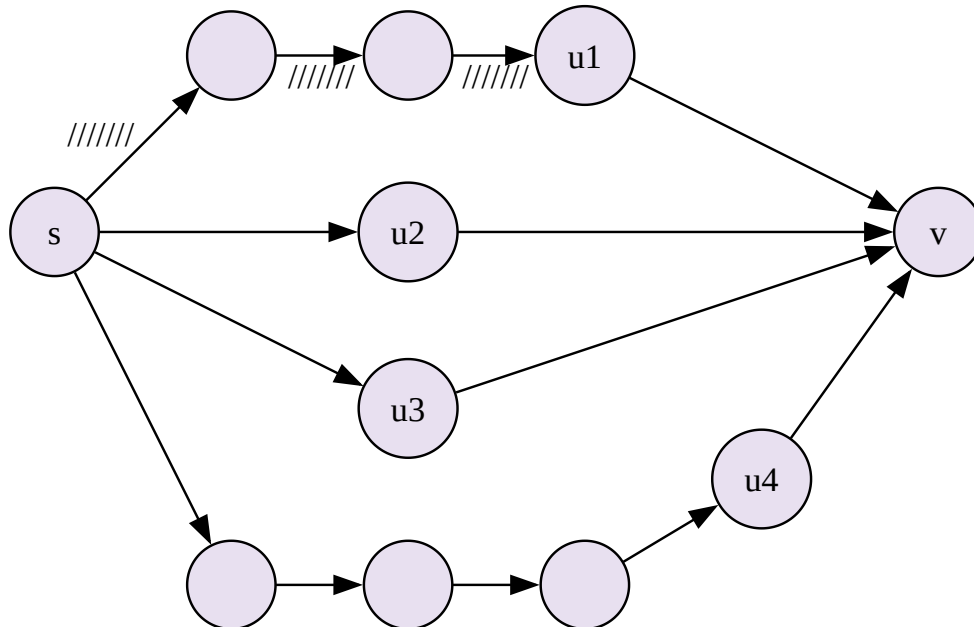
Dimostrazione:



$$\delta(s, v) = \begin{cases} +\infty & \text{se } s \text{ non è raggiunge } v \\ m & \text{n lung. di un cammino minimo da } s \text{ a } v \end{cases}$$

$$(u, v) \in E \implies \exists \text{ cammino da } s \text{ a } v \text{ di lunghezza } \delta(s, u) + 1 \implies \delta(s, v) \leq \delta(s, u) + 1$$

Esempio:



Base:

prima di v: $0 \implies v = s \quad d[s] = 0 = \delta(s, s)$ e quindi è sempre 0.

Passo induttivo:

Hp.Ind) $d[v] \geq \delta(s, v)$ se prima di v sono entrati in coda al più k-1 nodi.

f) $d[v] \geq \delta(s, v)$ se per ... K nodi prima di v sono entrati in coda k nodi $k > 0$, v entra in coda quando u è in testa e $(u, v) \in E$ e v è BIANCO

$$d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, u) \geq \delta(s, v)$$

Lemma:

Se durante BFS(G, s) vale che $Q = [v_1, v_2, \dots, v_r]$ (v_1 è il primo nodo in coda), allora:

1. $v_1, v_2, \dots, v_r$ sono GRIGI
2. $d[v_1] \leq d[v_2] \leq \dots \leq d[v_r] \leq d[v_r] + 1$

Dimostrazione:

Per induzione sul numero di operazioni effettuati su Q

base:

- 0 operazioni: $Q = \emptyset$
- 1 operazione: $Q = [s]$, s è grigio e $d[s] = 0$

Passo induttivo:

Hp.Ind) Valgono 1) e 2) se sono state effettuate al più k-1 operazioni su Q



f) Valgono 1) e 2) se sono state effettuate al più k operazioni su Q

Devo analizzare l'ultima operazione effettuata:

$$\frac{Q = [v_1, v_2, \dots, v_r]}{J \xrightarrow{\text{distanze}} j+1}$$

a) se l'operazione è Dequeue(Q)

$$Q = [\cancel{v_1}, v_2, \dots, v_r]$$

$$d = [\cancel{j}, j, j+1, j+1]$$

b) Se l'operazione è Enqueue(Q, v):

$$Q = [v_1, v_2, \dots, w]$$

$$d = [j, j, \dots, j+1, j+1]$$

Sto esaminando $Adj[v_1]$ e $v \in Adj[v_1]$ e w è BIANCO

Ho colorato w di grigio, gli ho assegnato $d[w] = d[v_1] + 1 = j+1$ e ho inserito w in coda

Dim correttezza:

BFS(G, s) termina:

- il ciclo while termina con $Q = \emptyset$
- inserisco in coda solo nodi BIANCHI, li coloro di GRIGIO prima di inserirli e al più li ricoloro BIANCHI
- Ci sono al più $|V|$ iterazioni del ciclo while (visto che a ogni iterazione del while estraggo un nodo da Q)

Fb) $d[v] = \delta(s, v)$ e $\pi[v]$ è lim pred. Di v si usa una connessione minima da s a v

- se s non raggiunge v : $d[v] = +\infty$, e $\pi[v] = NIL$. Nell'inizializzazione S non entra in coda.
- Se s raggiunge v : $V_k = v \in V | \delta(s, v) = k$

Per induzione su K si dimostra che se $v \in V_k$, esiste un'unica iterazione del while per cui:

- s viene colorato di GRIGIO
- $d[v]$ passa da $+\infty$ a k
- se $v \neq s$, $\pi[v]$ passa da NIL a $u \in V_{k-1}$
- v entra in coda

BASE:

$$k = 0$$

$V_0 = s$, s colorato di GRIGIO, $d[s] = 0$, s entra in coda prima della prima iterazione del while. Mai più modificati perchè s non torna BIANCO

**PASSO INDUTTIVO:****Hp.Ind)** 1) 2) 3) 4) $\forall v \in V_{k-1}$ **f)** 1) 2) 3) 4) $\forall v \in V_k$ $v \in V_k \quad \alpha(s, v) = k, \exists$ un cammino da s a v di lunghezza k :

- $s : v_1, v_2, \dots : v_{k-1}, v$
- $s \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_{k-1} \rightarrow v$ E non esiste un cammino più corto.

Supponiamo che $v_{k-1,1}$ sia il primo a entrare in Q tra i $v_{k-1,r}$
 è quindi anche il primo in testa alla coda Q .

$$Q = [v_{k-1,1} \dots]$$

$$v \in Adj[v_{k-1,1}] \quad v \text{ è BIANCO}$$

Questo è l'istante in cui v diventa GRIGIO, entra in coda, $d[v] = d[v_{k-1,1}] + 1$ e $\pi[v] = v_{k-1,1}$

Per ipotesi induttiva: $(k-1) + 1 = k$

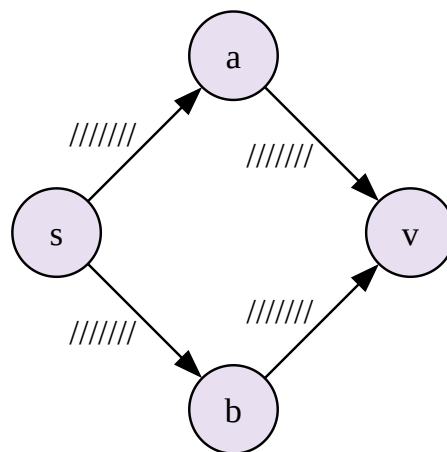
$d[v] = \delta(s, v) = k$, perchè $v \in V_k$. $\pi[v]$ è un pred. Di v su un cammino minimo.

Def:

(V, π) Albero di visita BFS da s ed è un albero dei cammini di **lunghezza minima da s**

Dato G ed s , l'albero è unico?

NO:



possiamo notare come non vi sia un percorso di lunghezza minima assoluta, perchè i due percorsi possibili si equivalgono (entrambi a 2 archi)

Esempio:

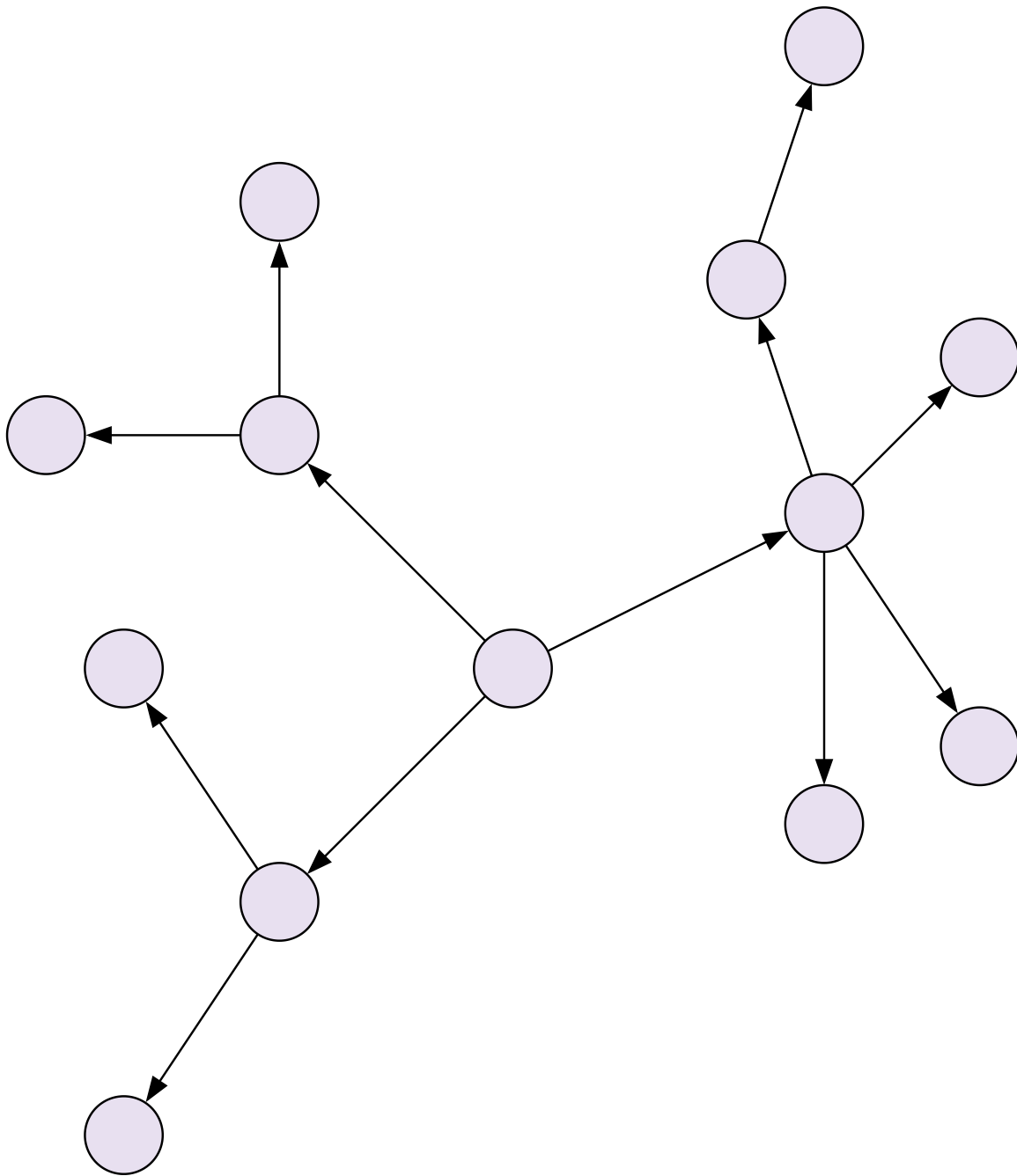
Dato G ed s trovare un nodo a **distanza** massima da s

- Eseguo BFS(G, s), poi cerco il max in d (molto inefficiente)
- Ogni volta che estraggo un nodo da Q , lo salvo, e all'ultima iterazione del while, quando Q è vuota, restituisco l'ultimo nodo estratto da Q

**Sfida:**

G è un grafo non orientato, connesso ($\forall u, v$ esiste un cammino tra u a v), aciclico (non ho nessuna situazione dove da un nodo posso raggiungere il medesimo nodo)

Determinare il Diametro di G (distanza massima tra due nodi), con un algoritmo efficiente



versione non efficiente (v^2):

```
max <- 0
for (each u e V){
  m <- BFS(G,u)
  if (m > max){
    max <- m
  }
}
```



Definizione:

$G = (V, E)$ è un albero (libero) se G è un grafo non orientato, connesso e aciclico

connesso $\forall u, v \in V, \exists$ un cammino da u a v

aciclico $\forall u, v \in V, \nexists$ un cammino *semplice* di lung > 0 da u a v

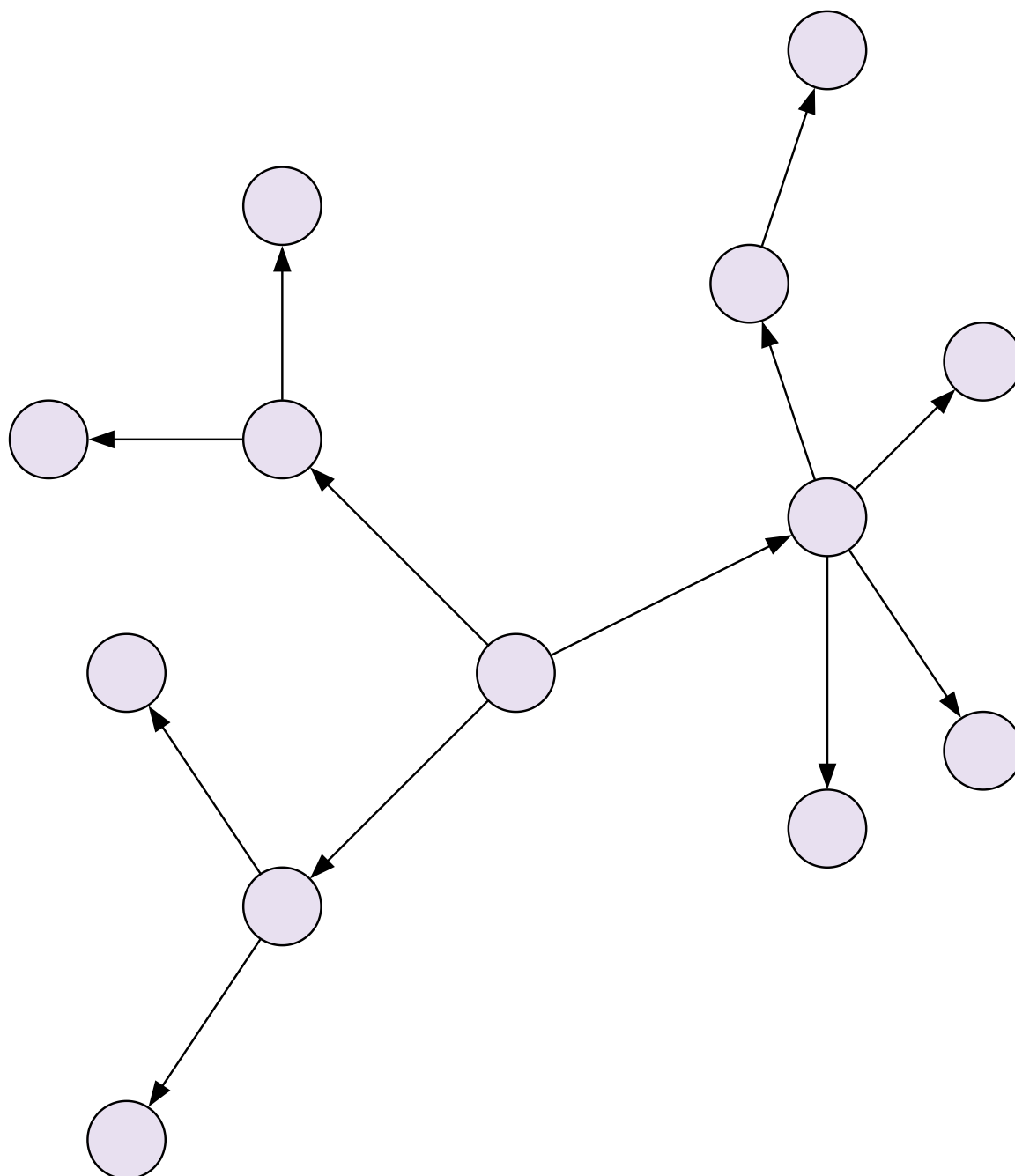
Un cammino semplice che usa al più una volta ogni arco

Proprietà:

Se $G = (V, E)$ è un grafo non orientato le seguenti affermazioni sono **equivalenti**:

1. G è un albero (connesso e aciclico)
2. G è connesso e rimuovendo un arco qualsiasi si disconnette
3. G è connesso a $\forall u, v \in V, \exists!$ un cammino semplice da u a v
4. G è aciclico e aggiungendo un arco qualsiasi diventa aciclico
5. G è connesso e $|E| = |V| - 1$
6. G è aciclico e $|E| = |V| - 1$

Gli alberi di cui stiamo parlando, sono di questo tipo:



dove non si sa chi sia il genitore e chi il figlio, ma appena scelgo una radice, immediatamente si capisce chi è il genitore e chi è il figlio, e quindi si capisce anche chi è il padre di chi, e così via. Quando si parla di albero libero, non si ha ancora decisa la radice.

Bisogna stare attenti a giocare bene quei $|V| - 1$ archi, perchè si rischia di sconnettere l'albero

Se aggiungo anche solo un arco, l'albero diventa ciclico. Allo stesso modo, se rimuovo anche solo un arco, l'albero diventa disconnesso.

1) $\Rightarrow$ 2)

hp) G è connesso e aciclico

f) G è connesso e rimuovendo un arco qualsiasi si disconnette

Se p.a. $\exists u, v \in E$ t.c. $G' = (V, E \setminus u, v)$ è disconnesso

Hplnd) con n nodi 4 implica 5



$\theta(|V|) \cdot O(|V| + |E|)$ con $|E| = |V| - 1 \implies O(|V|^2)$ perchè G è connesso e quindi deve visitare tutti i nodi

Se G è un albero BFS(G, s) costa $O(|V|)$, con liste di adiacenza.

Visita in profondità (DFS)

DFS visita in profondità

Problema (1):

Dato un grafo $G = (V, E)$ non orientato decidere se G è connesso

Eseguo BFS(G,1) e controllo sal termine che tutti i nodi sono diventati NERI. Costo $O(|V| + |E|)$

Problema (2):

Dato un grafo $G = (V, E)$ orientato decidere se G è fortemente connesso ($\forall u, v \in V, \exists u \rightsquigarrow v$)

Per fortemente connesso si intende che da ogni nodo posso raggiungere tutti gli altri nodi, e viceversa.

Per ogni $v \in V$ eseguo BFS(G, v) e controllo che tutti i nodi siano diventati NERI. Costo

$\theta(|V|) \cdot O(|V| + |E|)$

Problema (3):

Dato G orientato decidere se G è ciclico

Perchè abbiamo visto sia la visita in ampiezza che la visita in profondità?

DFS visita in profondità

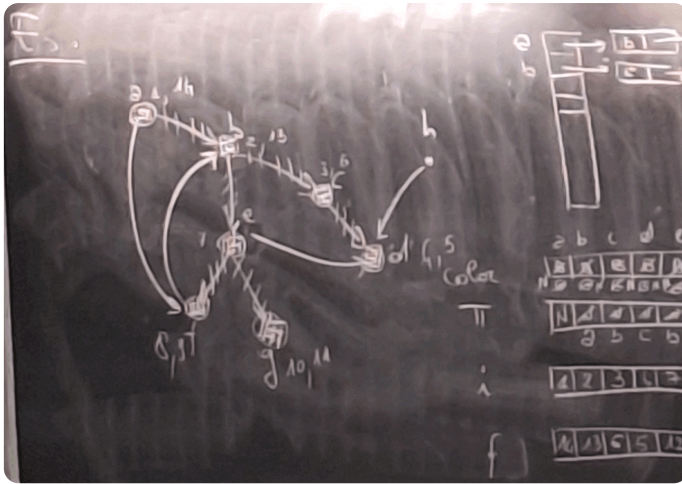
idea:

- Voglio visitare tutto G (ripete più volte)
- A partire da un nodo u cerco di esplorare in profondità un cammino che parte da u

Useremo

1. Colori: BIANCO, GRIGIO, NERO
2. Tempo TIME
3. i[u] tempo di fine visita di u
f[u] tempo di inizio visita di u
4. $\pi[u]$ predecessore di u nella visita

Esempio:



	a	b	c	d	f	g	h
color	B	B	B	B	B	B	B
π	N	/	/	/	/	/	/
$\rightarrow$		a	b	c	b	e	e
i	1	2	3	4	7	8	10
f	14	13	6	5	12	3	11

```

DFS(G){    # calcola le distanze tra i nodi
  for(each u in V){
    color[u] <- BIANCO # li coloro tutti di bianco nel caso ci siano
    variabili già inizializzate
    pi[u] <- NIL
  }
  TIME <- 1
  for(each u in V){
    if (color[u] == BIANCO){
      DFS_Visit(G,u)
    }
  }
}

```

```

DFS_Visit(G,u){
  color[u] <- GRIGIO
  TIME <- TIME + 1 # alla prof non piace la variabile globale, ma la tollera,
  quindi la passo come parametro
  i[u] <- TIME
  for(each v in Adj[u]){
    if (color[v] == BIANCO){
      pi[v] <- u
      DFS_Visit(G,v)
    }
  }
  color[u] <- NERO
  TIME <- TIME + 1
  f[u] <- TIME
}

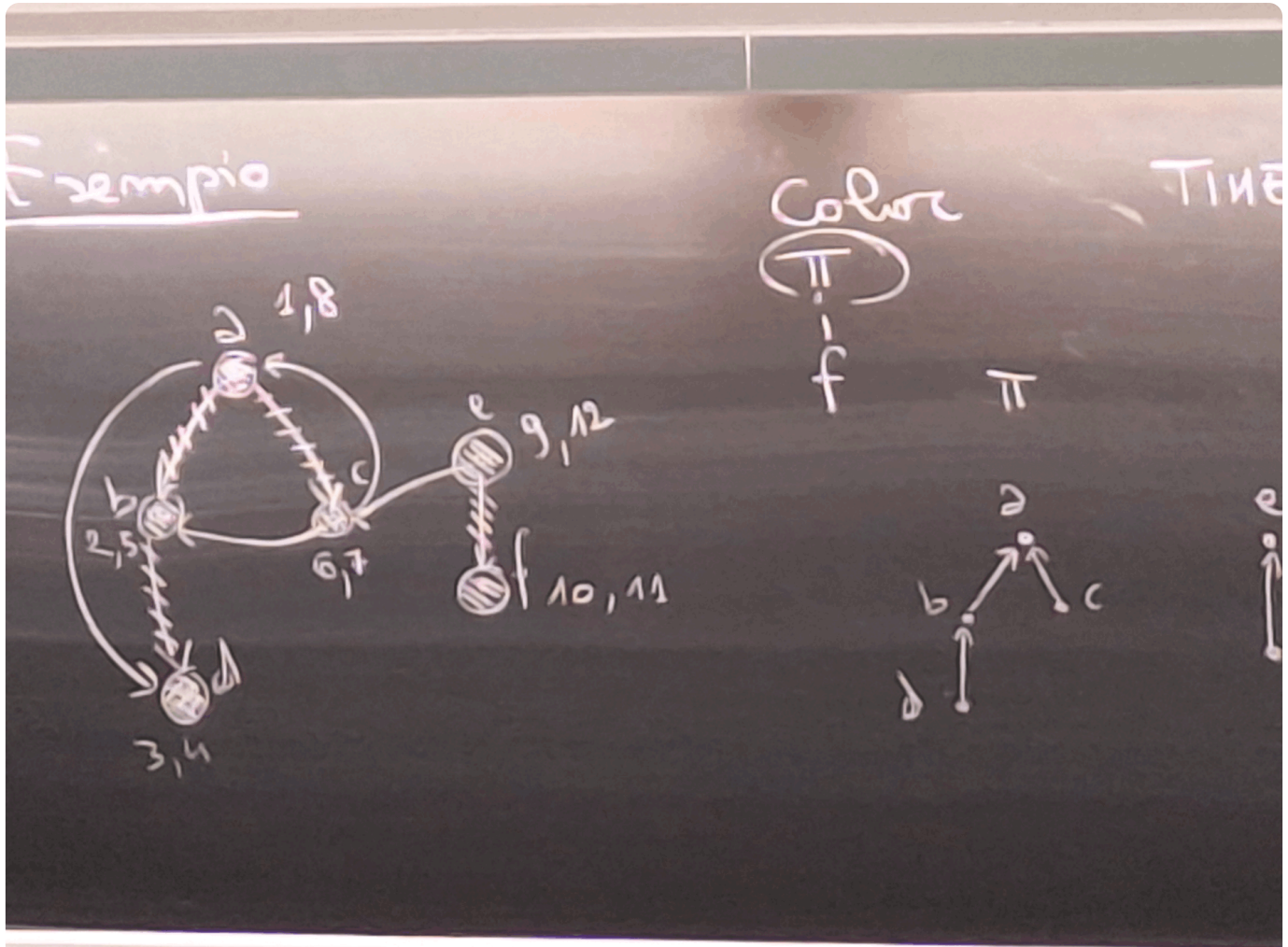
```

con costi:

Matrici di adiacenza

Liste di adiacenza

$$O(|V| + |V|^2) = \theta(|V|^2) \quad O(|V| + |E|)$$



Si noti come rifare il medesimo processo, se svolto nell'ordine alfabetico opposto, porterebbe a un albero di visita completamente diverso, e quindi a tempi di inizio e fine visita completamente diversi.

π memorizzata, la foresta di visite DFS,

Come posso capire se v è discendente di u nella foresta di visita DFS?

$s \rightarrow \cdot \rightarrow \cdot \rightarrow u$

scrivere codice per capire $\bar{u}$ secondo π

vettori i ad f :

$[0][1][2][\dots][\dots][8][9][\dots][2|v]$ TIME

$i[a] \rightarrow f[a]$ è l'intervallo di tempo in cui $\text{DFS_visit}(G, a)$ è nella pila delle chiamate ricorsive.

v è discendente di $u \iff [i[u], f[u]] \subseteq [i[v], f[v]]$

Teorema delle parentesi:

Sia $G = (V, E)$ un grafo.

Al termine di $\text{DFS}(G) \forall u, v \in V$ vale una delle seguenti:

1. $[i[v], f[v]] \subseteq [i[u], f[u]]$ oppure
2. $[i[u], f[u]] \subseteq [i[v], f[v]]$



$$3. [i[u], f[u]] \subseteq [i[v], f[v]] = \emptyset$$

Teorema dl cammino bianco:

Come posso capire se v è discendente di u nella foresta di visita DFS e voglio capirlo al tempo $i[u]$?

- Deve essere BIANCO
- Deve essere raggiungibile dal nodo u
- Il cammino tra u e v deve essere composto tutto da nodi BIANCHI

Sia $G = (V, E)$ un grafo e $u, v \in V$.

v è discendente di u nella foresta DFS $\iff$ al tempo $i[u]$ esiste un cammino di nodi BIANCHI da u a v

Dimostrazione

hp) u è collegato a v

Fs) Al tempo $i[u]$ esiste un cammino di nodi BIANCHI

Se considero $v, \pi[v], \pi[\pi[v]], \dots, \pi^k[v] = u$, da u a v , per forza erano tutti BIANCHI.

hp) Al tempo $i[u]$ esiste un cammino di nodi BIANCHI da u a v

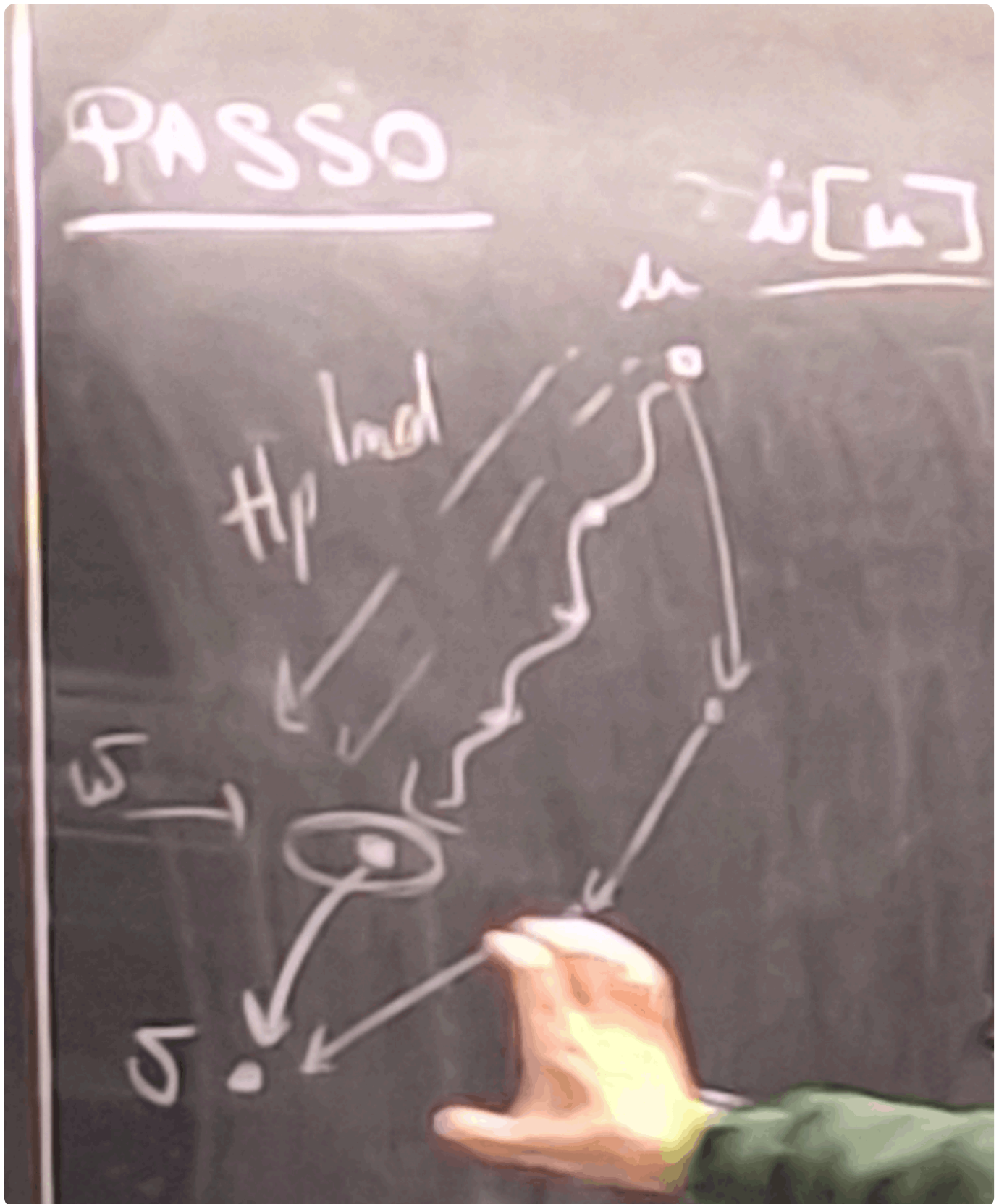
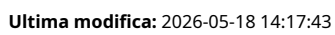
Fs) v è discendente di u nella foresta DFS

Per induzione sulla lunghezza: del più lungo cammino semplice da u a v di nodi BIANCHI

base:

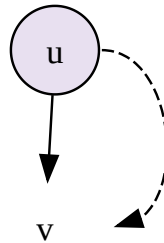
lunghezza = 1: $\pi[v] = u$

passo induttivo:

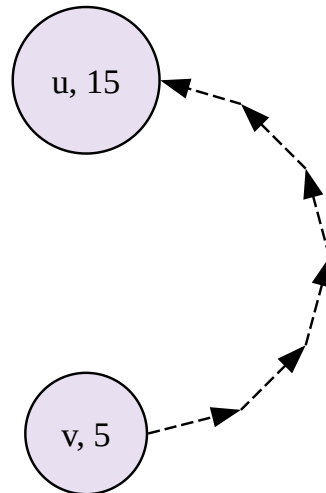


Classificazione DFS degli archi di un grafo

- ### 1. arco da GRIGIO a BIANCO



2. arco da GRIGIO a GRIGIO



albero di visite, nell'albero di visite parte da un nodo (u) a un suo predecessore (v)

Back Edge, o arco all'indietro collega un nodo u a un suo predecessore v nell'albero di visite DFS

3. arco da GRIGIO a NERO **Forward Edge** collega un nodo u a un suo successore, e si usa quando ci accorgiamo che c'è una scorciatoia tra due nodi già visitati ($i[u] < i[v]$) **Cross Edge**, archi di attraversamento collega sottoalberi di visite e collegano alberi di visita diversi (collega un punto a suo fratello) o due rami sullo stesso livello ($i[v] < i[u]$)

Come distinguo i forward edge dai cross edge?

Forward Edge	Cross Edge
$u[v[]]$	$v[], u[]$
$i[u] < i[v]$	$i[v] < i[u]$

Teorema:

$G = (V, E)$ è ciclico sse durante DFS(G) trovo almeno un arco da GRIGIO a GRIGIO (back edge)

Attenzione:

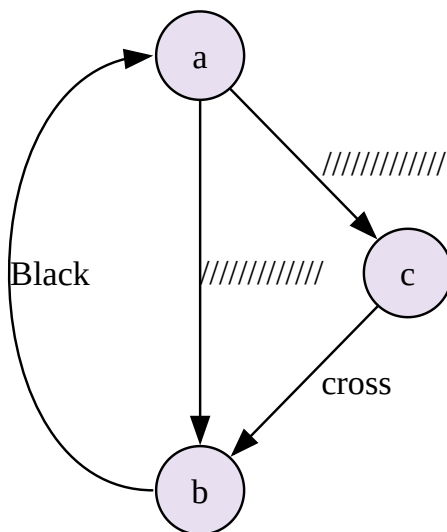
G non deve essere orientato, l'arco da b ad a non è un black edge perchè $\pi[b] = a$ quindi l'arco $\{b, a\}$ è un arco di visita

Domande:

- Posso contare i black edge durante DFS e inferire questi sono i cicli semplici del grafo?
 - I black edge sono al più $|E| \leq |V|^2$



- I cicli possono essere fino a $2^{|V|} - 1$ (grafo completo - caso in cui per stampare tutti i cicli ci serve tempo esponenziale)

Esempio:**Dimostrazione:**

$\Longleftrightarrow$) **hp)** G è ciclico

fs) almeno un BACK EDGE

Supponiamo che il primo nodo a diventa grigio in DFS.

Sia u_1

C'è almeno un cammino di Bianchi da u_1 a u_k $\xRightarrow{\text{Teo del cammino bianco}}$ u_k diventa discendente di u_1 nell'albero di visita DFS. $\implies$ quando scopro u_k e u_k diventa grigio, u_1 è ancora grigio. L'arco (u_k, u_1) è un back edge.

$\Longleftarrow$)

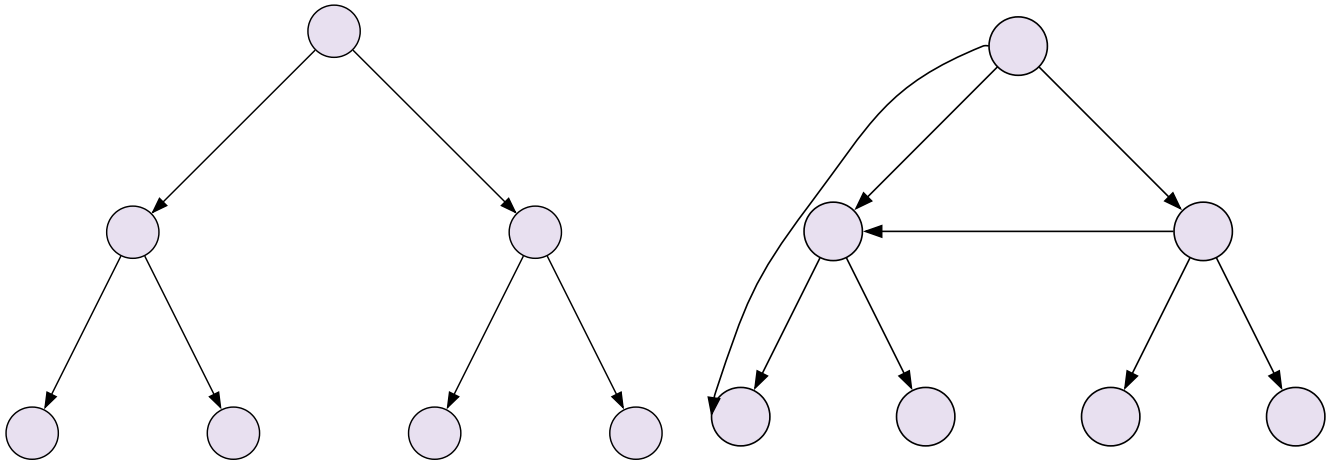
hp) durante DFS(G) trovo almeno un arco da GRIGIO a GRIGIO (back edge)

fs) G è ciclico

Il ciclo è formato dagli archi di visita che portano da u_1 a u_k + il black edge (u_k, u_1)

Esercizi:

- Modificare DFS in modo che quando viene trovato un Black edge, si stampi almeno un ciclo del grafo (attenzioni sui grafi non orientati)
- Scrivere il codice di una procedura che verifica se un grafo non orientato è un albero libero (si fa con una unica visita DFS)

**Alberi****Grafi orientati aciclici**

Se parto dalla radice, sono tutti **archi di visita**

basta una ricorsione, con il caso base le foglie.
Posso quindi lavorare per induzione

(con grafici ciclici manca il caso base)
I casi base sono i nodi foglia che non hanno
archi uscenti, detti anche **nodi pozzo**

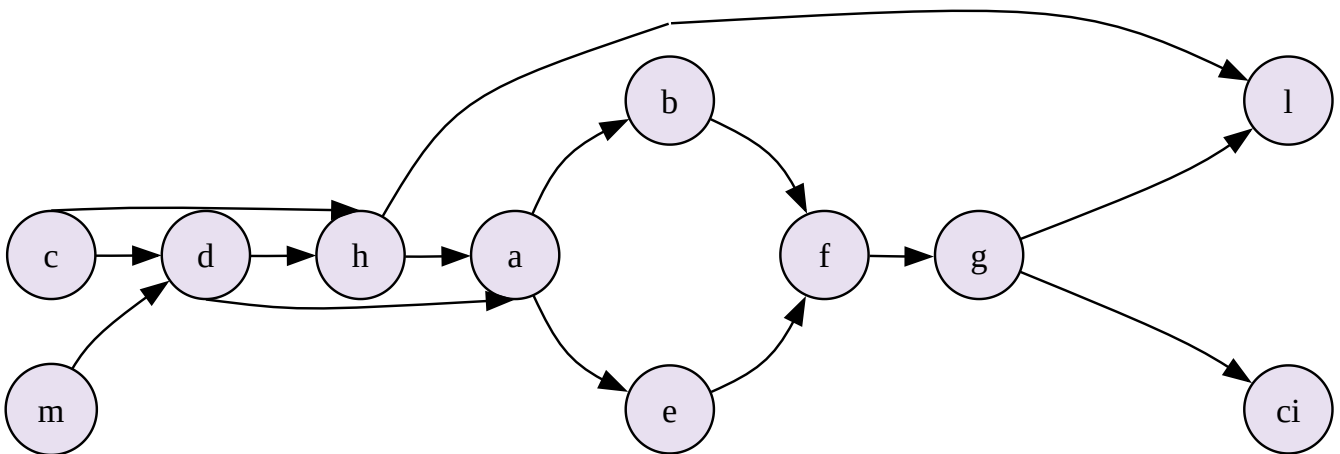
Definire *un* possibile ordinamento sui nodi di un grafo orientato aciclico (**DAG**)

Nell'ordinamento i nodi pozzo (casi base nella ricorsione) devono avvenire per ultimi.

Def ORDINAMENTO TOPOLOGICO

Dato un DAG $G = (V, E)$ un ordinamento topologico è un ordinamento $u_1 < u_2 < \dots < u_n$ dei nodi di G tale che se $(u, v) \in E$, allora $u < v$ nell'ordinamento.

Esempio:



$a < b < c < d < e < f < g < h < i < l < m$ è un ordinamento topologico di G
NO $(l, i) \in E$ ma

Dato $G = (V, E)$ DAG come calcolo un topological sort di G ?

Come riconosco i nodi pozzo in DFS?

Diventano NERI per primi.

Modifico DFS_visit (parte ricorsiva) e quando un nodo diventa NERO, lo inserisco in testa a una lista



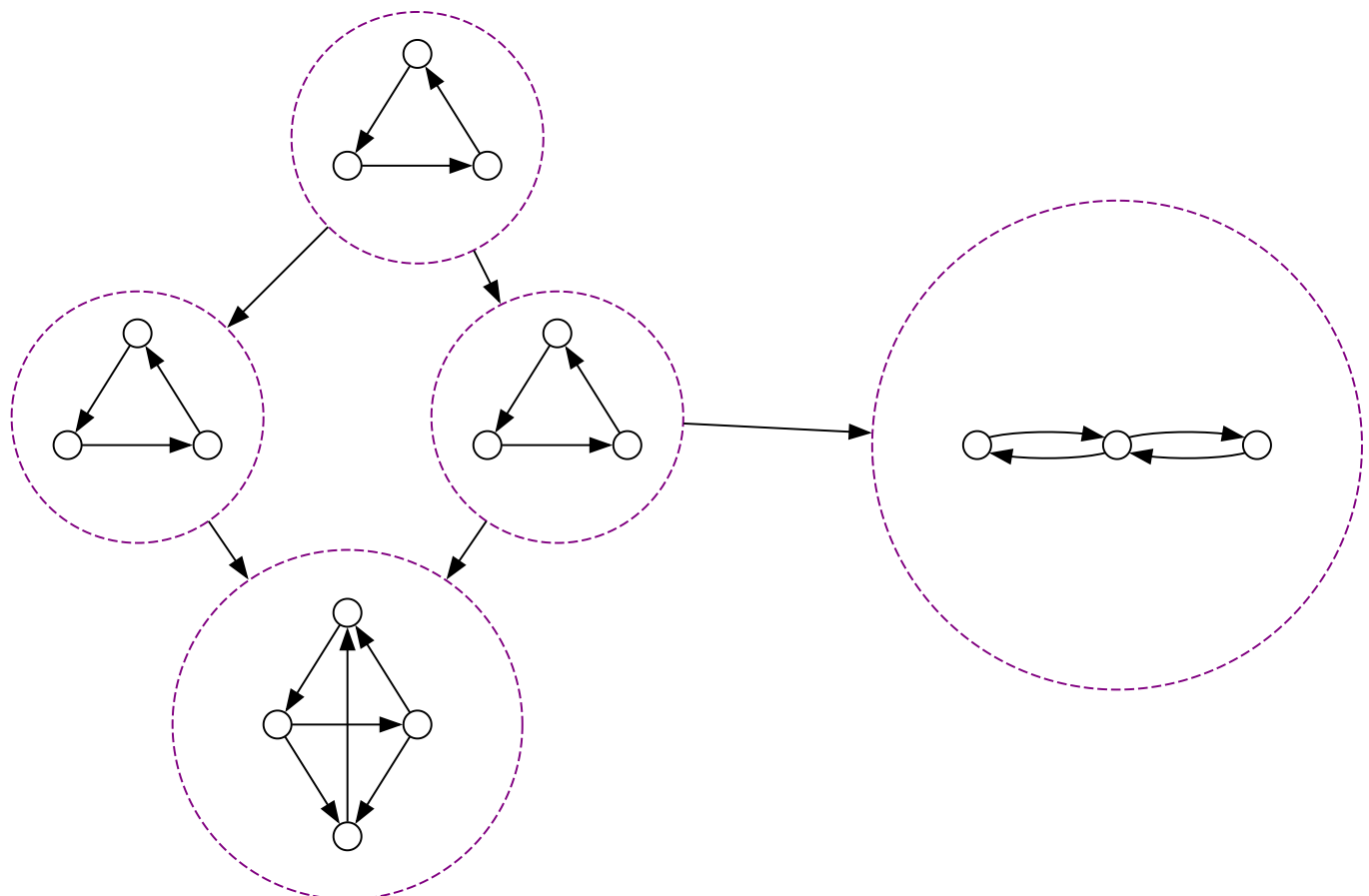
```
DFS{
  for....
  L <- NIL
  for...{
    DFS_visit(G,u)
  }
  return L    # L è la lista che contiene i nodi di G ordinati topologicamente
}

DFS_visit(G,u){
  ...
  for (each v in Adj[u]){
    ...
  }
  color[u] <- NERO
  inserisci_in_testa(L,u)    # parte aggiunta per ordinamento topologico
}
```

Se G non è un DAG?

Avremo una struttura con dei cicli. Possiamo però considerare i singoli cicli, come dei macronodi, e quindi considerare il grafo come se fosse un DAG, e quindi applicare l'ordinamento topologico.

Componenti fortemente connesse



Def:

Sia $G = (V, E)$ un grafo orientato, una componente fortemente connessa C di G (strongly connected



component) è un sottoinsieme di nodi di $G(C \subseteq V)$ massimale di nodi mutualmente raggiungibili

- $C \subseteq V$
- $\forall u, v \in C, \exists$ un cammino da u a v e un cammino da v a u
 - $\exists$ un cammino da v a u_1
- $\forall w \notin C \forall u \in C [\nexists \text{ cammino da } w \text{ a } u]$
- $\forall w \notin C \forall u \in C [\nexists \text{ cammino da } u \text{ a } w]$

osservazione:

$C_1, C_2, \dots, C_k$ sono le componenti fortemente connesse di G formano una partizione di V

è importante che gli archi siano ordinati, per poi mettere ogni elemento in testa alla lista ogni volta che diventano NERI, cosa che accade quando gli viene attribuito il tempo di fine visita. se un grafo non avesse archi, avremmo $n!$ possibili ordinamenti topologici.

come calcolarle ?

Dato u calcolare la componente fortemente connessa in cui si trova u ? (S.C.C(u))

$v \in S.C.C(u) \iff u \rightsquigarrow v \text{ e } v \rightsquigarrow u$	Devo trovare tutti i raggiungibili da u Forw(u) Devo trovare tutti i raggiungibili da v Back(v)
$S.C.C(u) = Forw_G(u) \cap Back_G(u)$	BFS(G, u) I nodi neri al termine sono il Forw(u) $BFS(G^{-1}, u)$ I nodi neri al termine sono il Back(u)

$$\theta(|V| + |E|)$$

Elimino da G S.C.C(u) itero sui nodi rimasti

.-->.-->.-->(.)-->.

$$O(|V|) \cdot O(|V| + |E|)$$

Le comp. fort. connesse di G e G^{-1} coincidono.

Problema:

Voglio calcolare tutte le comp. fort. connesse di G in $\theta(|V| + |E|)$

Eseguo DFS(G)

Ricordiamo che il grafo delle S.C.C. è un DAG

Come identifico un algoritmo un nodo di S.C.C. pozzo?

è il primo a diventare NERO

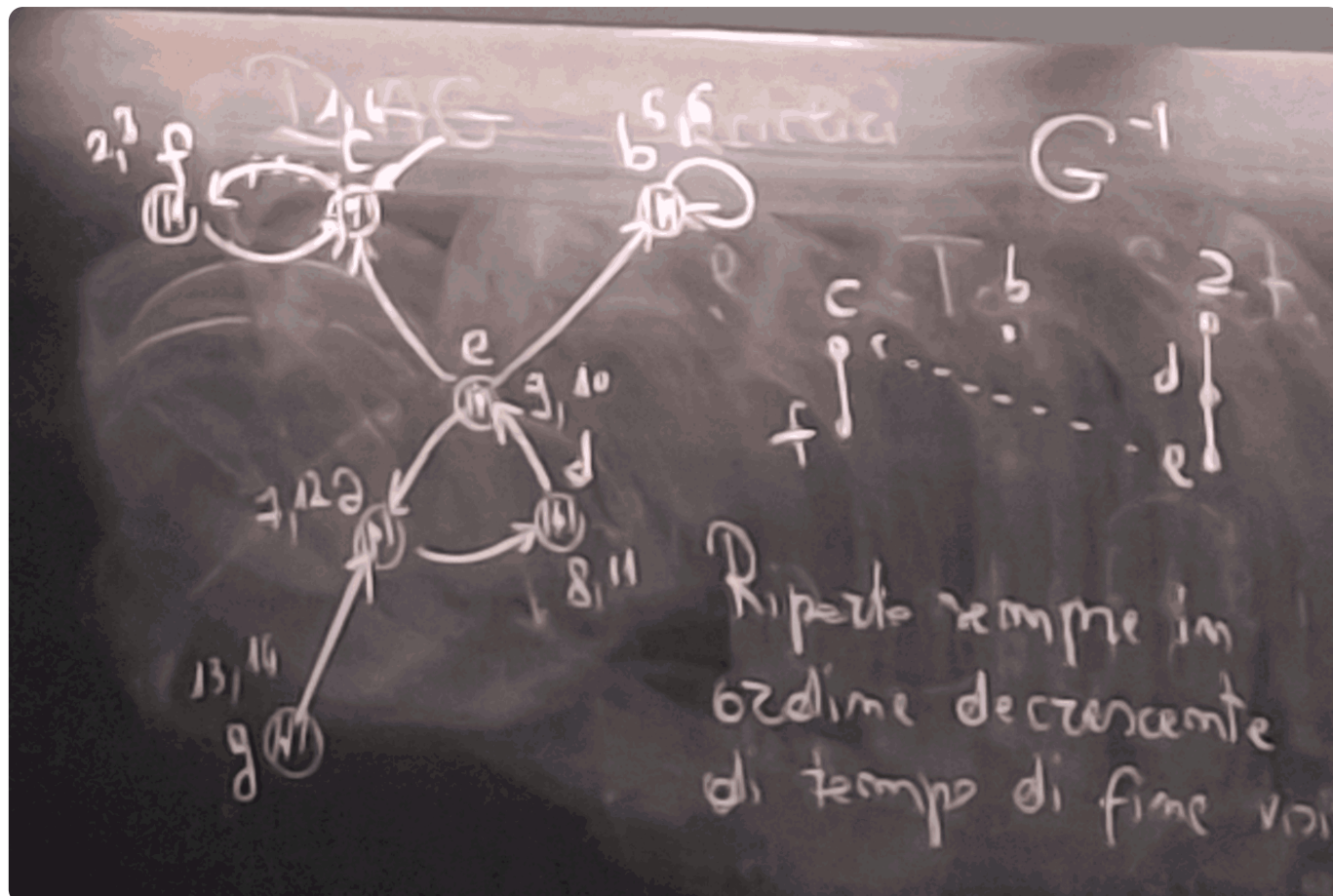
Come identificare almeno un nodo di una S.C.C. sorgente?

L'ultimo a diventare NERO, Il tempo di fine visita mancante

Esempio:

Il nodo u con tempo di fine visita massimo è in una S.C.C. sorgente in G , di conseguenza u è in una S.C.C. pozzo in G^{-1}

Se in $DFS(G^{-1})$ parto da u il primo albero che costruisco è S.C.C.(u)



Riparto sempre in ordine decrescente di tempo di fine visita

Algoritmo di Kosaraju per le S.C.C.

- Eseguo DFS(G)
 - Calcolo G^{-1}
 - Eseguo DFS(G^{-1}) procedendo nel ciclo principale in ordine decrescente di tempo di fine visita
- Gli alberi della seconda DFS sono le S.C.C.
 Gli archi che collegano alberi di visita distinti sono gli archi del grafo delle S.C.C.

$$\theta(|V| + |E|)$$

In alternativa posso usare l'algoritmo di Tarjan.

usa una sola DFS(G) e una coda aggiuntiva.

è più complesso da implementare, ma è più efficiente in pratica.

Esercizio:

Calcolare il grafo chiusura transitiva (G^*)

$$u \rightsquigarrow v \text{ in } G \implies u \rightarrow v \text{ in } G^*$$

$$M_G + M_G \cdot M_G + \dots + (M_G)^i + \dots + (M_G)^{|V|-1}$$

1. Se u e v sono nella stessa S.C.C., $u \rightarrow v$ in G^*
2. Se u e v sono in 2 S.C.C. distinte, $u \rightarrow v$ in G^* $u \rightarrow v$ in G^*

Devo trovare un modo eff. per calcolare G^* nel caso che G sia un DAG (Il DAG verrà applicato al



grafo delle S.C.C. di G)

Cercare di sfruttare topological sort

Es.:

Sia $G = (V, E)$ orientato, i cui nodi sono colorati con verde e rosso.

1. decidere se in G esiste un ciclo contenente solo nodi rossi
2. decidere se in G c'è un ciclo contenente almeno un nodo rosso
3. eseguo DFS(G) usando solo i nodi rossi, c'è un ciclo di rossi sse trovo un back edge.

Stampo in out un ciclo

$$O(|V| + |E|)$$

4. Trovo le S.C.C.

Controllo che ci sia almeno una S.C.C. contenente un nodo rosso e avente almeno 2 nodi oppure avente un solo nodo con un self loop.

$$O(|V| + |E|)$$



Chiusura transitiva

$G = (V, E)$ orientato

$G^* = (V, E^*) \quad (u, v) \in E^* \iff u \rightsquigarrow v \text{ in } G$

$G^{Scc} = (V^{Scc}, E^{Scc})$

$$(u, v) \in E^* \iff (C_u, C_v) \in (E^{Scc})^*$$

1. calcolo G^{Scc}

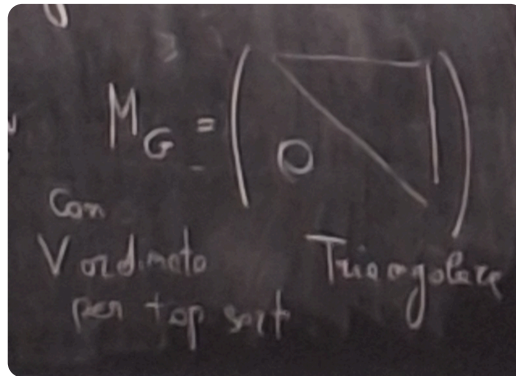
2. calcolo $(G^{Scc})^* \longrightarrow$ è un DAG

Mi serve un algoritmo eff. per calcolare la chiusura transitiva di un DAG

Sia $G = (V, F)$ un DAG, calcolare G^*

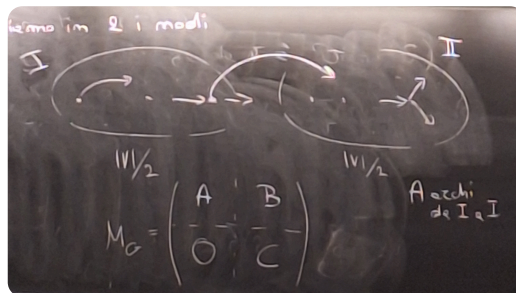
1. Ordino i nodi di G con topological sort

$M_G =$



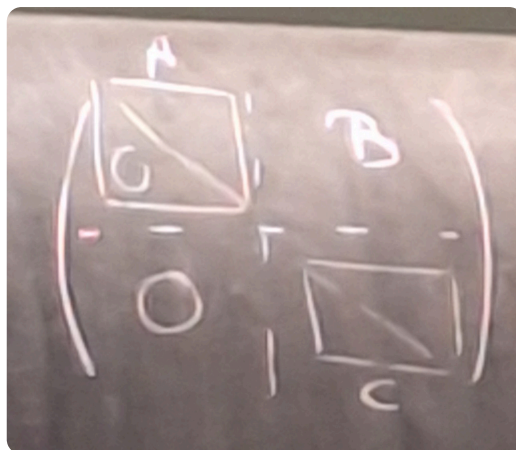
con una matrice Triangolare.

Dividiamo in 2 i nodi (I e II)



Guardiamo la matrice M_G detta A , con archi da I a I. chiamiamo una matrice B con archi da I a II, una matrice 0 senza archi e una matrice C con archi da II a II.

ancora una volta si tratta di triangolari superiori



$$T(n) = \begin{cases} \theta(1) & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + \theta(n^{2/3}) & \text{else} \end{cases}$$

$$\implies T(n) = \theta(n^{2/3})$$

Da Prodotto tra matrici

$$a \ G^*$$

$$A, B \quad \text{calcolabile} \quad A \cdot B$$

$$H = \begin{matrix} & lol & A & 0 \\ 0 & & lol & B \\ 0 & 0 & & lol \end{matrix}$$

$$(H)^* = \begin{matrix} & lol & A & A \cdot B \\ 0 & & lol & B \\ 0 & 0 & & lol \end{matrix}$$

Questa è una relazione Binaria.

$$f : R \rightarrow R$$



Grafi Pesati

$G = (V, E, W)$, dove $W : E \rightarrow \mathbb{R}^+$

Sui grafi pesati vedremo 2 cose:

1. Albero minimo di copertura
 - Algoritmo di Kruskal
 - Algoritmo di Prim
2. Cammino minimo da s a v
 - Algoritmo di Dijkstra
 - Algoritmo di Floyd-Warshall

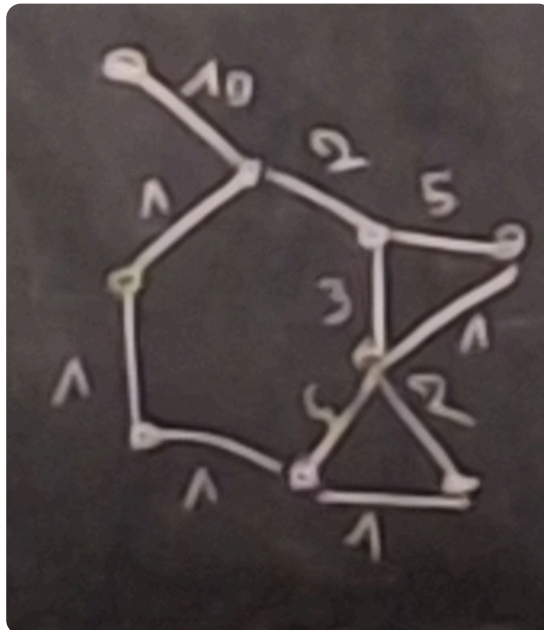
è considerato **gravissimo** confondere i 2 problemi, con bocciatura istantanea all'orale.

Albero minimo di copertura

Detto anche *Minimum Spanning Tree - MST*

$G = (V, E, W)$ pesato, non orientato e connesso, con i pesi positivi.

problema;



In un albero di questo tipo, chiamo (V, T) albero di copertura di $G = (V, E, W)$ se (V, T) è un albero

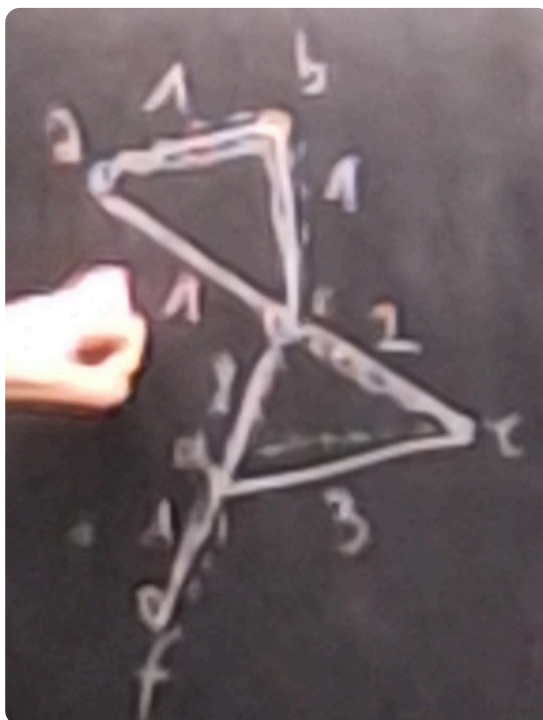




(V, T) è un albero di copertura di $G \rightarrow$ è un albero di copertura e il suo peso è minimo tra i pesi degli alberi di copertura dove il peso di V, T è

$$W(V, T) = \sum_{\{u, v\} \in T} W(\{u, v\})$$

Esempio:



E quindi $T_2: W(T_2) = 7$ non è unico

Una applicazione di questo principio è quello di avere diverse città da collegare con delle strade, ma voler usare meno asfalto possibile, e quindi cercare di minimizzare il peso totale delle strade da costruire.

O in un sistema informatico, dove i nodi sono dei server e gli archi sono dei cavi, e voglio collegare tutti i server con il minor costo possibile.

$G=(V,E,W)$ sono arioentati. connessi, pesati con $W : E \rightarrow \mathbb{R}^{>0}$

Trovare un albero di compertura di peso minimo

Sottografi di copertura $E' \subseteq E$

t.c. $G = (V, E')$ è connesso

problemno:

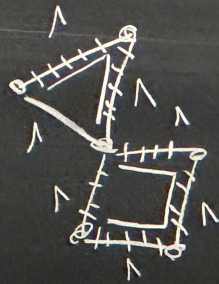
Dato $G = (V, E, W')$, trovare un sottografo G' di copertura aventa peso minimo tra tutti i sottografi di copertura di G

La soluzione è sempre e comunque un albero di copertura di G

esempio



La soluzione è un albero di copertura



... G' $W(G') = 6$
 — T albero $W(T) = 5$

Idea: Voglio costruire T

- $A \leftarrow \emptyset$
 $A \subseteq E$
- Ad ogni passo trovo $\{u, v\}$ t.c. $A \cup \{\{u, v\}\} \subseteq T'$ con T' M.S.T
- Aggiungo l'arco ad A :
 $A \leftarrow A \cup \{\{u, v\}\}$

esempio



$a = \emptyset \rightarrow A = \{\{a, b\}\} \rightarrow A = \{\{a, b\}, \{e, d\}\} \rightarrow A = \{\{a, b\}, \{e, d\}, \{b, c\}\}$
 e come albero finale: $A = \{\{a, b\}, \{e, d\}, \{b, c\}, \{c, d\}\}$
 oppure: $A = \{\{a, b\}, \{e, d\}, \{b, c\}, \{c, d\}, \{c, f\}\}$

Devo trovare "un arco safe per A "

Def:

Sia $A \subseteq T$ con T u.s.t di G $\{u, v\} \in E$ è **SAFE** per A se $\exists T'$ M.S.T t.c. $A \cup \{\{u, v\}\} \subseteq T'$

Algoritmo MST



```

A <- []
for each ({u,v} in E){
  if ({u,v} è SAFE per A){
    A <- A U {u,v}
  }
}

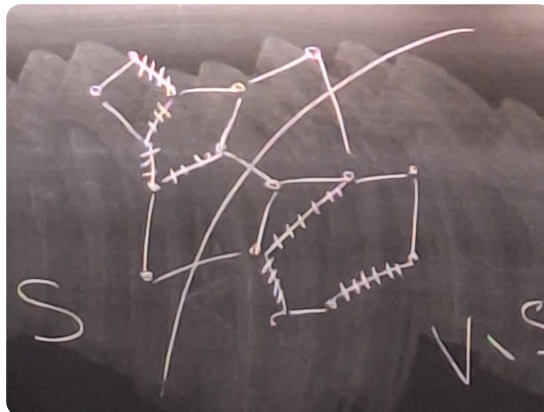
```

Dobbiamo trovare delle condizioni che assicurino che $\{u, v\}$ sia SAFE per A senza dover colorare un UST T'

Avremo 2 risultati $\rightsquigarrow$ quando 2 algoritmi:

- Teo $\rightsquigarrow$ PRIM
- Corollario $\rightsquigarrow$ KRUSKAL

Quando inizio a costruire l'albero, inizialmente avrò degli alberi separati, perchè non ho nessun arco che va da sinistra a destra. Per questo le definizioni che verranno introdotte ora serviranno a descrivere questo fenomeno e a comprenderlo meglio.



Def:

- $(S, V \setminus S)$ è un **taglio** (cut) per G (partizione di V in 2 blocchi)
- $\{u, v\}$ è attraversa il taglio $(S, V \setminus S)$ se
 $u \in S$ e $v \in V \setminus S$ oppure
 $u \in V \setminus S$ e $v \in S$
- $A \subseteq E$ rispetto il taglio $(S, V \setminus S)$ se
 $\forall \{u, v\} \in A \quad \{u, v\}$ non attraversa il taglio $(S, V \setminus S)$
- $\{u, v\}$ è **leggero** per il taglio $(S, V \setminus S)$ se $\{u, v\}$ attraversa il taglio $(S, V \setminus S)$ e tra tutti gli archi che attraversano $(S, V \setminus S)$ $\{u, v\}$ ha peso minimo

Teorema:

Sia $G = (V, E, W)$ un grafo connesso, non orientato, con pesi positivi.

Sia T M.S.T. di G , e sia $A \subseteq T$

Sia $(S, V \setminus S)$ un taglio t.c. A rispetto al taglio $(S, V \setminus S)$

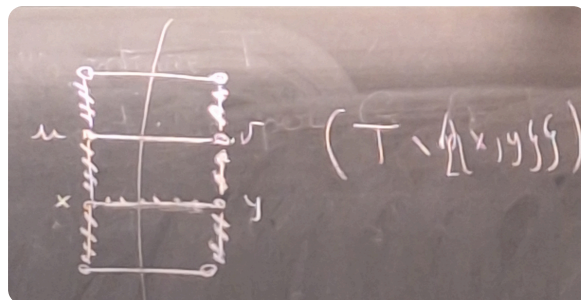
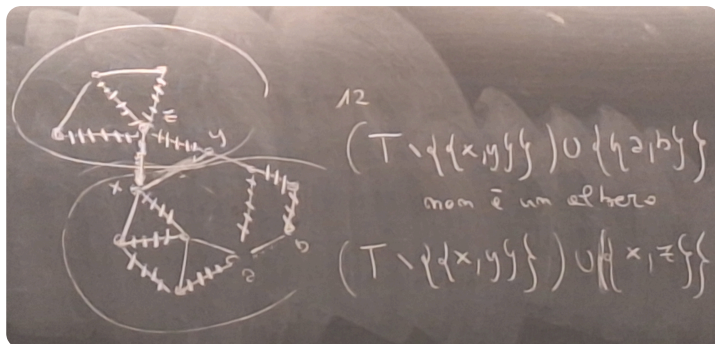
Sia $\{u, v\}$ un arco leggero per il taglio $(S, V \setminus S)$



Allora L'arco $\{u, v\}$ è SAFE per A

$$\exists T' \text{ M.S.T di } G \text{ t.c. } A \cup \{\{u, v\}\} \subseteq T'$$

Lemma:



Se $T \subseteq E$ è uno spanning tree di $G = (V, E)$ e $\{u, v\} \notin E \setminus T$ e $\{x, y\} \in T$ e $\{x, y\}$ sta sul cammino da u a v in T , allora:

$$T' = T \setminus \{\{x, y\}\} \cup \{\{u, v\}\} \text{ è uno spanning tree di } G$$

Dimostrazione Lemma:

Ts) T è un albero

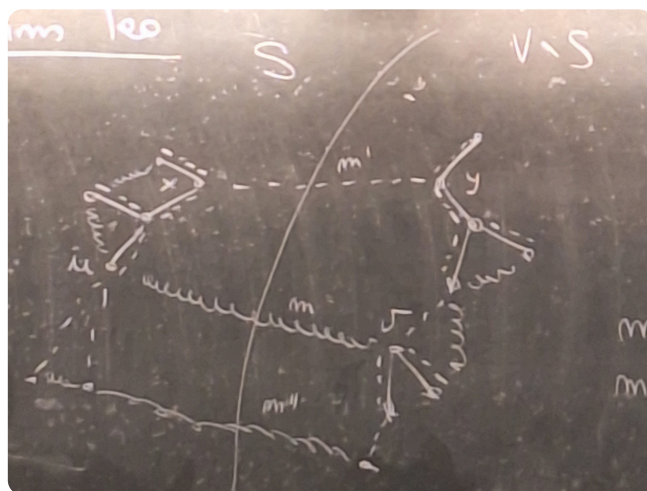
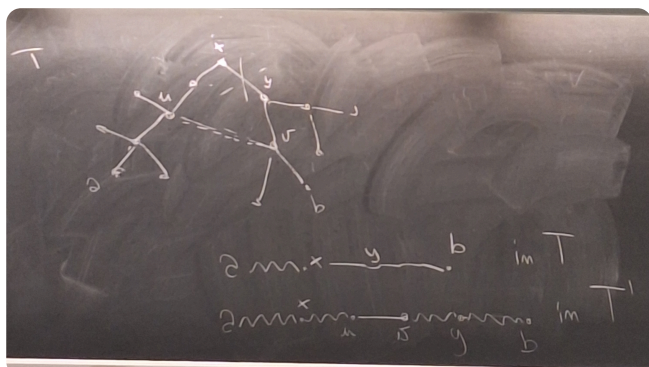
Tseq) $|T'| = |V| - 1$ e T' è connesso

$$|T'| = |T| - 1 + 1 = |V| - 1 + 1 - 1$$

T' è connesso

Ts) $\forall a, b \in V \exists$ un cammino da a a b in T'

Dimostrazione Teorema:



considero:

$$A \cup \{\{u, v\}\}$$

Ts) T' M.S.T. t.c. $A \cup \{\{u, v\}\} \subseteq T'$



1. Se $\{u, v\} \in T$ $A \cup \{\{u, v\}\} \subseteq T'$
2. Se $\{u, v\} \notin T$ $A \cup \{\{u, v\}\} \subseteq T$

Sia $\{x, y\}$ un arco di T sul cammino da u a v in T , t.c. $\{x, y\}$ attraversa il taglio $(S, V \setminus S)$

$$T' = (T \setminus \{\{x, y\}\}) \cup \{\{u, v\}\} \implies$$

$\implies$ Dal lemma T' è un albero di copertura

TS) M.S.T.

$$W(T') = W(T) - w(\{x, y\}) + w(\{u, v\})$$

$$= W(T) - m' + m \leq W(T)$$

$\implies$ quindi T' è un M.S.T.

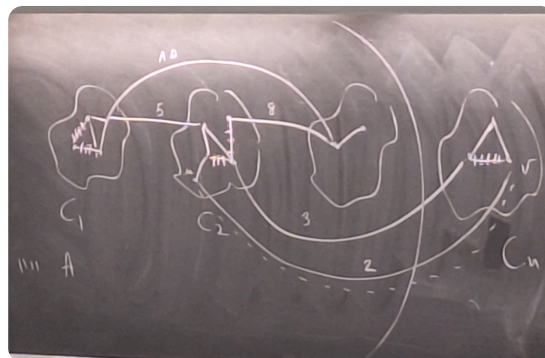
Corollario:

Sia $\{C_1, C_2, \dots, C_m\}$ una partizione di V

Sia $A \subseteq T$ con T M.S.T. t.c. $\forall \{a, b\} \in B$ se $a \in C$: allora $b \in C$

Tra tutti gli archi che collegano nodi che si trovano in blocchi diversi, sia $\{u, v\}$ un arco di peso minimo

$\{u, v\}$ è SAFE per A



Algoritmo di Kruskal

Idea:

1) All'inizio

$A \leftarrow \emptyset$

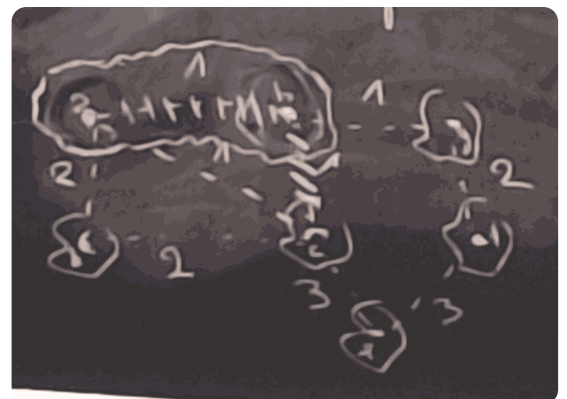
ogni nodo sta in una componente separata

Aggiungo ad A un arco di peso minimo di G

$A = \{\{a, b\}\}$

$A = \{\{a, b\}, \{b, c\}\}$

$A = \{\{a, b\}, \{b, c\}, \{g, f\}\}$

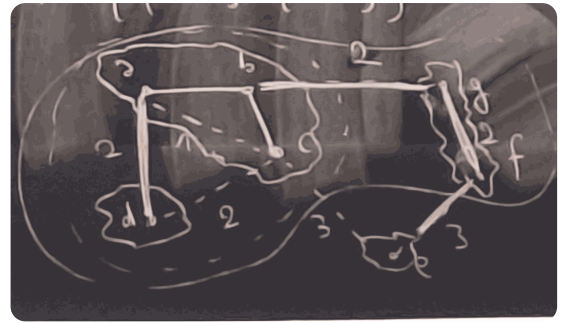


Per implementare Kruskal in modo efficiente:

- Analizzo gli archi



- Ordino gli archi in ordine crescente di peso
- Una struttura dati efficiente per gestire le componenti connesse
 - All'inizio ogni nodo è una componente separata
 - Ad ogni passo devo trovare se un arco collega o meno componenti distinte
 - Se aggiungo un arco che collega 2 componenti distinte, fondo le due componenti



Ci servirà una struttura ad insiemi disgiunti.

```

$$$ \   $$$ \
$$$ | $$$ |
$$$ |$$$ /  $$$$$$ \ $$$ \ $$$ \ $$$$$$ \ $$$ | $$$ \ $$$$$$ \ $$$ |
$$$$$ /  $$$ _$$$ \ $$$ | $$$ |$$$ _$$$ |$$$ | $$$ | \_$$$ \ $$$ |
$$$ $$$< $$$ | \_$$$ |$$$ | $$$ |$$$$$$$ \ $$$$$$ / $$$$$$ |$$$ |
$$$ |$$$ \ $$$ | $$$ | $$$ | \_$$$ \ $$$ _$$$< $$$ _$$$ |$$$ |
$$$ | \$$$ \ $$$ | \$$$$$$$ |$$$$$$$ |$$$ | \$$$ \$$$$$$$ |$$$ |
\_| \_| \_| \_| \_$$$ / \_$$$ / \_| \_| \_$$$ \_| \_|

```

```

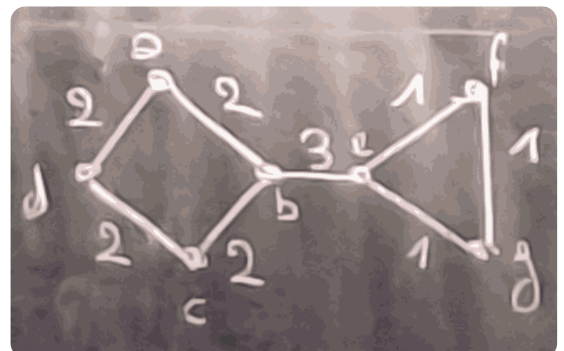
Kruskal(G=(v,E,W)){
  Sort(E) # ordino gli archi in ordine crescente di peso
  A <- []
  for (each v in u){
    Make(v) # ogni nodo è una componente separata
  }
  for (each {u,v} in E){ # in ordine di peso
    if (Find(u) != Find(v)){ # se u e v sono in componenti distinte
      A <- A U {u,v} # aggiungo l'arco ad A
      Union(u,v) # fondo le due componenti
    }
  }
  return A
}

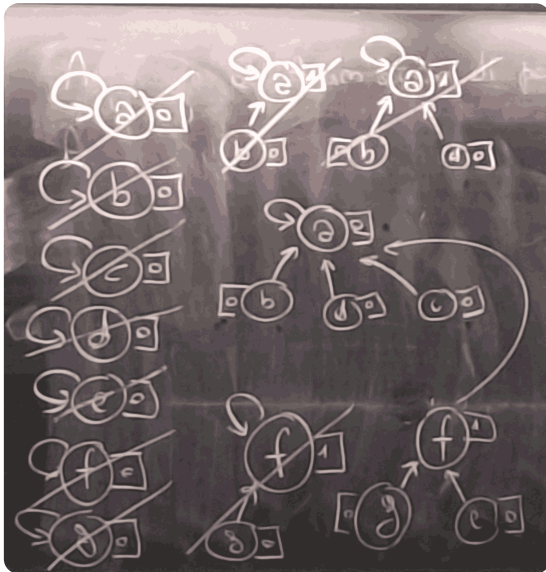
```

<i>b</i>	<i>f</i>	...
<i>e</i>	<i>g</i>	...
3	1	...



<i>f</i>	<i>f</i>	<i>g</i>	<i>a</i>	<i>a</i>	<i>d</i>	<i>c</i>	<i>b</i>
<i>g</i>	<i>e</i>	<i>e</i>	<i>b</i>	<i>d</i>	<i>c</i>	<i>b</i>	<i>e</i>
1	1	1	2	2	2	2	3





$$A = \emptyset$$

$$A = \{\{f, g\}\}$$

$$A = \{\{f, g\}, \{f, e\}\}$$

$$A = \{\{f, g\}, \{f, e\}, \{a, b\}\}$$

L'ordine dipenderà anche dalle varie condizioni che abbiamo posto per l'ordinazione (in questo caso usiamo l'ordine alfabetico)

$$A = \{\{f, g\}, \{f, e\}, \{a, b\}, \{a, d\}\}$$

$$A = \{\{f, g\}, \{f, e\}, \{a, b\}, \{a, d\}, \{d, c\}\}$$

$$A = \{\{...\}...\{b, e\}\}$$

Complessità:

- Sorting: $O(|E| \log |E|)$
 - al più: $O(|E| \log |V|)$
- Make: $O(|V|)$
- Find: $\theta(|E|)$
- Union: $\theta(|V|)$

Tenendo conto che find e union vengono eseguiti più volte (il sorting invece è eseguito una sola volta, come il make):

- Find e Union nel tempo: $O(|E| \alpha(|V|, |E|))$

Costo Totale:

$O(|E| \log |V|) + O(|E| \alpha(|V|, |E|))$, e quindi la prima parte è dominante, e quindi

$$O(|E| \log |V|)$$

Esercizio:

Rifare i conti nel caso di insiemi disgiunti gestiti con:

1. liste concatenate
2. Liste concatenate con weighted union
3. Alberi
4. Alberi con union by rank

Esercizio

Trovare ipotesi sui G che permettono di usare Counting Sort (Bucket Sort)

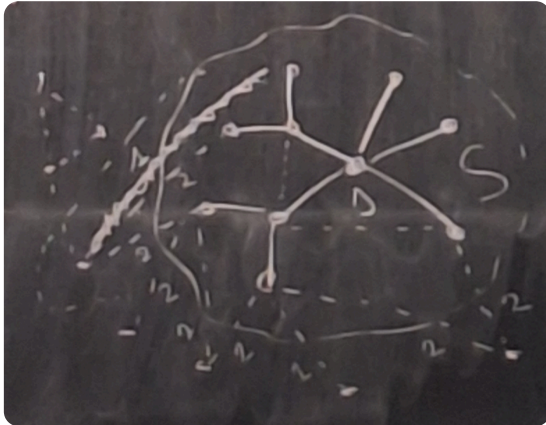
Correttezza:

$A \subseteq T$ con T M.S.T.

perchè ad ogni passo aggiungo ad A un arco SAFE in base al corollario



Algoritmo di Prim



Idea:

Ad ogni passo A è un albero con altezza radice a

Aggiungo un arco leggero che collega un nuovo nodo all'albero

Strutture Dati:

Per ogni nodo u memorizzo quale è l'arco più leggero che mi permette di collegarlo ad A

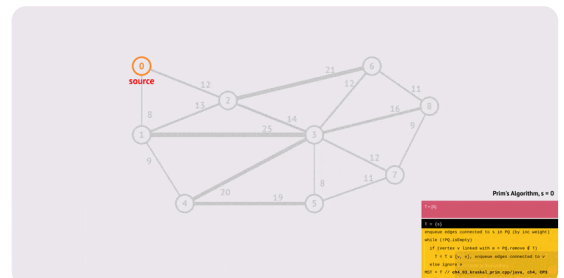
```
key[u] # peso dell'arco
pi[u]  # nodo di partenza dell'arco
```

Ad ogni passo tra tutti i nodi ancora non collegati ad A scelgo di collegare:

- quello con **key** minima
- Lo collego usando l'arco $\{\pi[u], u\}$
- Scandisco **Adj[u]** e modifico le **key** e i π dei nodi raggiunti da u (se necessario)

In ogni istante collego al M.S.T. che sto costruendo il nodo che ha chiave minima. In quell'istante esploro la sua lista di adiacenza per verificare se ci sono chiavi da modificare (ad esempio, nei primi casi si passerebbe da $key[b] = +\infty \rightarrow key[b] = 1$).

Le chiavi possono solo diminuire il loro valore, per questo serve una Min Heap.



Sito usato per l'animazione: visualgo.net

Per coloro che visualizzano in PDF, link per l'animazione disponibile a: [il mio Git](#) (non garantisco il supporto per gli anni a venire)

```

$$$$$$$ \          $$ \
$$  _$$$ \        \__|
$$ |  $$ | $$$$$$ \  $$ \ $$$$$$ \ $$$$$$ \
$$$$$$$ |$$  _$$$ \  $$ |$$  _$$$  _$$$ \
$$  _$$ /  $$ |  \__| $$ |$$  /  $$ /  $$ |
$$ |      $$ |      $$ |$$ |  $$ |  $$ |
$$ |      $$ |      $$ |$$ |  $$ |  $$ |

```



```

\__|      \__|      \__|\__| \__| \__|

Prim(G=(V,E,W),s){
  for (each u in V){
    key[u] <- +infinity
    pi[u] <- NIL
  }
  key[s] <- 0
  Q <- V # Min Heap ordinato per key
  Build_Min_Heap(Q)
  while(Q != empty){
    u <- Extract_Min(Q)
    for (each v in Adj[u]){
      if (v in Q && key[v] > W({u,v})){
        key[v] <- W({u,v})
        pi[v] <- u
        Decrease_Key(Q,v) # Ripristino le min heap
      }
    }
  }
}

```

1. Dove è memorizzato il M.S.T?

Nel vettore pi

2. Perché serve $v \in Q$ Per non creare cicli

3. correttezza:

Teorema del taglio ($Q, V \setminus Q$)

4. Complessità:

- per il for: $\theta(|V|)$
- Da key a Build_Min_Heap: $\theta(|V|)$
- per il while:
 - prima parte: $\theta(|V| \log |V|)$
 - Decrease_Key: $O(|E| \log |V|)$
 - complessità totale: $O(|E| \log |V|)$

Esercizio:

Valutare la complessità senza usare la mean heap

Cammini minimi - di peso minimo

avendo $G = (V, E, W)$ (orientato o non orientato) pesato, con $W : E \rightarrow \mathbb{R}^+$

cammino: Sequenza di archi consecutivi

$$u_0 \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k$$

Peso di un cammino: (con k peso di un arco)

$$W(u_0 \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_k) = \sum_{i=1}^k k_i$$



Single Source Shortest Path

Dato $s \in V \forall u \in V$, calcolare un cammino di peso minimo da s a u

Abbiamo già visto un algoritmo simile (BFS), ma funziona solo con cammini di lunghezza minima da S

- $W : E \rightarrow \mathbb{R}^+ \quad \forall (u, v) \in E \quad W((u, v)) = 1$
Analogo se $\forall (u, v) \in E, \quad W((u, v)) = k$ costante, con un costo di $O(|V| + |E|)$ e utilizza come strutture:
- **colori**: per evitare di visitare più volte lo stesso nodo
- π : per memorizzare la distanza minima da S
- d : per memorizzare il nodo precedente nel cammino minimo da S a u
- **coda**: per gestire i nodi da visitare

Il problema è che la coda in DFS bastava per le sue necessità, perchè se guardavo le distanze di questi nodi, vi era un blocco di distanza h , e poi un blocco di distanza $h+1$, e così via, quindi non c'era bisogno di una struttura dati più complessa, perchè i cammini successivi erano sempre maggiori o uguali a quelli precedenti

Dovremmo cambiare la coda in una Min Heap per risolvere il problema.

```

$$$$$$$ \  $$ \      $$ \                $$ \
$$  _$$ \  \_ |      $$  |                $$  |
$$ |  $$ |$$ \  $$ \  $$ |  $$ \  $$$$$$ \  $$$$$$ \  $$$$$$ \  $$$$$$ \
$$ |  $$ |$$ | \_ |$$ |  $$ |  $$  _ _ _ _ | \_ $$ _ |  $$  _$$ \  \_ _ _ $$ \
$$ |  $$ |$$ |$$ \  $$$$$$ /  \ $$$$$$ \  $$ |  $$ |  \_ | $$$$$$ |
$$ |  $$ |$$ |$$ |$$ _$$ <  \_ _ _ $$ \  $$ |$$ \  $$ |  $$  _$$ |
$$$$$$$ |$$ |$$ |$$ |  \$$ \  $$$$$$ |  \$$$$$ |$$ |  \$$$$$ |
\_ _ _ _ /  \_ |$$ | \_ |  \_ | \_ _ _ _ /  \_ _ _ /  \_ |  \_ _ _ _ |
      $$ \  $$ |
      \$$$$$ |
      \_ _ _ /

```

```

Dijkstra(G=(V,E,W),s){ # W pesi positivi
  for(each v in V){
    pi[v] <- NIL
    d[v] <- +infinity
  }
  dw[s] <- 0
  Q <- V
  Build_Min_Heap(Q) # ordinato rispetto al valore dw
  while(Q != empty){
    u <- Extract_Min(Q)
    for (each v in Adj[u]){
      if (v in Q && dw[v] > dw[u] + W((u,v))){
        dw[v] <- dw[u] + W((u,v))
        pi[v] <- u
        Decrease_Key(Q,v) # Ripristino la min heap
      }
    }
  }
}

```



}

Nota: l'algoritmo funzionerebbe anche senza `v in Q` ma è stato inserito per evitare dubbi (al contrario di Prim, dove è fondamentale)

Correttezza:

1. Se ho un cammino di peso minimo da s ad a e da a a b , unendoli ho un cammino di peso minimo da s a b ?
NO
2. Se ho un cammino di peso minimo da s a b e a è un nodo sul cammino, allora i sottocammini da s ad a e da a a b sono di peso minimo?
SI

Complessità:

- per il for: $\theta(|V|)$
- While: $(O(|E|\log|V|))$
Totale $O(|E|\log|V|)$

Se usassimo Q come un vettore ordinato, seppur più semplice da scrivere, avremmo una complessità di $O(|V|^2)$, che è peggio di quella di Prim, quindi è importante usare la Min Heap. **tuttavia**, avendo fatto sparire $|E|$, potrebbe essere vantaggiosa se il grafo è molto denso.

All Pairs Shortest Path

$\forall u, v \in V$ calcola un cammino di peso minimo da u a v

Questo si vede subito nei navigatori (caso più semplice), ma ci sono anche dati più complessi, come una rete di consegne, perchè sia consumi e percorrenze cambiano nel giro di una giornata, con algoritmi estremamente più sofisticati di quelli che vedremo

ci sono due soluzioni:

```
for(each u in V){    # restituisce due vettori di vettori (due matrici)
    Dijkstra(G,u)    # -> pi[u][v] dw [u][v]
}
```

Complessità:

- $O(|V| \cdot |E|\log|V|)$ con Min Heap
- $O(|V|^3)$ con vettore ordinato

Algoritmo di Floyd-Warshall

**idea:**

Alla i -esima iterazione, costruisco solo cammini di cui nodi interni sono compresi tra 1 e $i-1$

$i = 1 \rightarrow$ nodi interni tra 1 e 0 $\rightarrow$ nessun nodo interno: $[1, 0] = \emptyset$

i generico: $j \in [1, i-1]$: tra peso minimo tutti i cammini da u a v con nodi interni in $[1, i-1]$

Come aggiorno il risultato al passo $i + 1$?

- nodi interni $[1, i]$
- i compare al più una volta, con nomi compresi tra 1 e $i-1$, da confrontare con il costo minimo di u e v , calcolato al passo precedente, Tenere il minimo

DW

- $DW_1 = W$ matrice dei pesi $+\infty$ se non l'arco

$$DW_{i+1} = \min(DW_i[w, v], DW_i[u, i] + DW_i[i, v])$$

```

$$$$$$$$\ $$\
$$  _____|$$ |
$$ |      $$ | $$$$$$\ $$\  $$\  $$$$$$$$ |
$$$$$\  $$ |$$  _$$\ $$ |  $$ |$$  _$$ |
$$  _|  $$ |$$ /  $$ |$$ |  $$ |$$ /  $$ |
$$ |      $$ |$$ |  $$ |$$ |  $$ |$$ |  $$ |
$$ |      $$ |\$$$$$$$ |\$$$$$$$ |\$$$$$$$ |
\_|      \_| \_____/  \_____$ $ | \_____/
                        $$\  $$ |
                        \$$$$$ |
                        \_____/

```

```

Floyd_Warshall(G){
  for(each u in V){
    for (each v in V){
      if ((u,v) == E){
        DW[u,v] <- W((u,v))
      } else {
        DW[u,v] <- +infinity
      }
    }
  }
  for (i <- 2 to |V| + 1){
    for (each u in V){
      for (each v in V){
        DW[u,v] <- min(DW[u,v], DW[u,i+1] + DW[i-1,v]) # uso il nodo i
        come ponte
      }
    }
  }
}

```

complessità: $\theta(|V|^3)$

In termini di spazio non è il massimo, perchè tiene la matrice in memoria, ma è un algoritmo molto semplice da implementare, e quindi è usato in molti casi, anche se non è il più efficiente (quello più efficiente lo vediamo alla magistrale).

